

SOLIDWORKS API (.NET VBA C#) — Examples (12/50):

- Create_Folding_Tool_Feature_Example_VBNE-
- Create_Machining_Costing_Analyses_Example_VBNE-

SOLIDWORKS API Documentation (converted from CHM)

Create Forming Tool Feature Example (VB.NET)

This example shows how to create a forming tool feature.

* Preconditions:

- * 1. Open `public_documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\sheetmetal\formtools\cover.sldprt`.
- * 2. On the SOLIDWORKS Design toolbar, click **Tools > Macro > New** and create a VB.NET macro.
- * 3. Copy and paste the code below into `SolidWorksMacro.vb`.
- * 4. Modify the two formtoolpath locations.
- * 5. Run the macro.
- * 6. At the pause, inspect the FeatureManager design tree and the graphics area for the drafted rectangular emboss forming tool.
- * Then select a flat face where to re-position (and replace) the forming tool.
- * 7. Press F5 finish.

*

* Postconditions:

- * 1. A dimple emboss replaces the drafted rectangular emboss.
- * 2. Inspect the FeatureManager design tree, graphics area, and the Immediate window.

*

* NOTE: Do not save the part as it may be used elsewhere.

*

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
```

```
    Dim featMgr As FeatureManager
```

```
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
```

```
        'Select a flat face where to insert an emboss forming tool
```

```
        boolstatus = swModel.Extension.SelectByRay(0.0553557395141979, 0.0303381093343091, 0.0266948404126879, -0.707061513640545, -0.415946801562901, -0.57188484347632, 0.000532844353755021, 2, False, 0, 0)
```

```
        featMgr = swModel.FeatureManager
```

```
        Dim LibFormToolFeatureData As LibraryFormToolFeatureData
```

```
        LibFormToolFeatureData = featMgr.CreateDefinition(swFeatureNameID_e.swFmFormToolInstance)
```

```
        LibFormToolFeatureData.LinkToFormTool = True
```

```
        LibFormToolFeatureData.FormToolPath = "E:\images\2025_SP02_release64\d250219.026.F63\WinRel64\WinRel64\data\Design Library\forming tools\embosses\drafted rectangular emboss.sldftp"
```

```
        Call LibFormToolFeatureData.OverrideDocumentSettings(True, True, True, True)
```

```
        LibFormToolFeatureData.RotationAngle = 0.5
```

```
        Dim swFeat As Feature
```

```
        swFeat = featMgr.CreateFeature(LibFormToolFeatureData)
```

```
        Stop ' Inspect the graphics area and FeatureManager design tree; select a flat face where to re-position (and replace) the forming tool
```

```
        Dim selMgr As SelectionMgr
```

```
        selMgr = swModel.SelectionManager
```

```
        Dim face2 As Face2
```

```
        face2 = selMgr.GetSelectedObject6(1, -1)
```

```
        Dim pickPt As Object
```

```
        pickPt = selMgr.GetSelectionPoint2(1, -1)
```

```

Dim MU As MathUtility
MU = swApp.GetMathUtility()

Dim mathPt As MathPoint
mathPt = MU.CreatePoint(pickPt)

swModel.ClearSelection2(True)

Dim swLibFormFeatData1 As LibraryFormToolFeatureData
swLibFormFeatData1 = swFeat.GetDefinition()

Debug.Print(swLibFormFeatData1.GetPunchID())
Debug.Print(swLibFormFeatData1.FormToolConfiguration)
Debug.Print(swLibFormFeatData1.FormToolPath)
Debug.Print(swLibFormFeatData1.LinkToFormTool)
Debug.Print(swLibFormFeatData1.RotationAngle)

Debug.Print(swLibFormFeatData1.AccessSelections(swModel, Nothing))

Dim face As Face2
face = swLibFormFeatData1.PlacementFace

swLibFormFeatData1.RotationAngle = 0.7833
Call swLibFormFeatData1.OverrideDocumentSettings(True, True, True, True)
swLibFormFeatData1.FormToolConfiguration = "Default"
swLibFormFeatData1.FormToolPath = "E:\images\2025_SP02_release64\d250219.026.F63\WinRel64\WinRel64\data\Design
Library\forming tools\embosses\dimple.SLDFTP"
swLibFormFeatData1.LinkToFormTool = True
Dim ent As Entity
ent = face
Call ent.Select4(True, Nothing)

Call swLibFormFeatData1.SetPlacementFace(face2, mathPt)

Debug.Print(swFeat.ModifyDefinition(swLibFormFeatData1, swModel, Nothing))

Dim face1 As Face2
face1 = swLibFormFeatData1.PlacementFace

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Create Full Symmetrical Angular Dimension Example (C#)

This example shows how to create a full symmetrical angular dimension between a centerline and a line.

```
//-----
// Preconditions: Verify that the specified part document template
// exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Opens a sketch.
// 3. Creates a centerline and three lines in
// the open sketch.
// 4. Selects the centerline and one of the lines.
// 5. Suppresses the dimension dialog.
// 6. Creates a full symmetrical angular dimension for
// the entities selected in step 4.
// 7. Unsuppresses the dimension dialog.
// 8. Examine the graphics area to verify step 6.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace AddSymmetricDimensionCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            bool status = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);

            //Sketch a centerline and three lines
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchMgr.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCenterLine(0.0, 0.043667, 0.0, 0.0, -0.050556, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(-0.102, 0.039667, 0.0, -0.086223, 0.011, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.142445, 0.066556, 0.0, 0.100889, -0.032333, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.085334, 0.036556, 0.0, 0.049658, -0.048341, 0.0);

            //Select the centerline and one of the lines
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", -0.00022223294397278, 0.0223334410869282, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0.113556103437018, -0.00144445141358242, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);

            //Suppress the dimension dialog box
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swInputDimValOnCreate, false);

            //Create a full symmetrical angular dimension
            status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", -0.00022223294397278, 0.0223334410869282, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0.113556103437018, -0.00144445141358242, 0, true, 0, null, 0);
            swDisplayDimension = (DisplayDimension)swModelDocExt.AddSymmetricDimension(0.0832913738352659, 0.112403597688285, 0);
            swModel.ClearSelection2(true);

            //Unsuppress the dimension dialog box
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swInputDimValOnCreate, true);

            swModel.ViewZoomtofit2();
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Create Full Symmetrical Angular Dimension Example (VBA)

This example shows how to create a full symmetrical angular dimension between a centerline and a line.

```
'-----
' Preconditions: Verify that the specified part document template
' exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Opens a sketch.
' 3. Creates a centerline and three lines in
'    the open sketch.
' 4. Selects the centerline and one of the lines.
' 5. Suppresses the dimension dialog.
' 6. Creates a full symmetrical angular dimension for
'    the entities selected in step 4.
' 7. Unsuppresses the dimension dialog.
' 8. Examine the graphics area to verify step 6.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)

    'Sketch a centerline and three lines
    Set swSketchMgr = swModel.SketchManager
    swSketchMgr.InsertSketch (True)
    Set swSketchSegment = swSketchMgr.CreateCenterLine(0#, 0.043667, 0#, 0#, -0.050556, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.102, 0.039667, 0#, -0.086223, 0.011, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(0.142445, 0.066556, 0#, 0.100889, -0.032333, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(0.085334, 0.036556, 0#, 0.049658, -0.048341, 0#)

    'Select the centerline and one of the lines
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", -2.2223294397278E-04, 2.23334410869282E-02, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0.113556103437018, -1.44445141358242E-03, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True

    'Suppress the dimension dialog box
    swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swInputDimValOnCreate, False

    'Create a full symmetrical angular dimension
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", -2.2223294397278E-04, 2.23334410869282E-02, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0.113556103437018, -1.44445141358242E-03, 0, True, 0, Nothing, 0)
    Set swDisplayDimension = swModelDocExt.AddSymmetricDimension(8.32913738352659E-02, 0.112403597688285, 0)
    swModel.ClearSelection2 True

    'Unsuppress the dimension dialog box
    swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swInputDimValOnCreate, True

    swModel.ViewZoomtofit2

End Sub
```

Create Full Symmetrical Angular Dimension Example (VB.NET)

This example shows how to create a full symmetrical angular dimension between a centerline and a line.

```

-----
Preconditions: Verify that the specified part document template
exists.
Postconditions:
1. Opens a new part document.
2. Opens a sketch.
3. Creates a centerline and three lines in
the open sketch.
4. Selects the centerline and one of the lines.
5. Suppresses the dimension dialog.
6. Creates a full symmetrical angular dimension for
the entities selected in step 4.
7. Unsuppresses the dimension dialog.
8. Examine the graphics area to verify step 6.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swModelDocExt As ModelDocExtension
        Dim swDisplayDimension As DisplayDimension
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)

        'Sketch a centerline and three lines
        swSketchMgr = swModel.SketchManager
        swSketchMgr.InsertSketch(True)
        swSketchSegment = swSketchMgr.CreateCenterLine(0.0#, 0.043667, 0.0#, 0.0#, -0.050556, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(-0.102, 0.039667, 0.0#, -0.086223, 0.011, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(0.142445, 0.066556, 0.0#, 0.100889, -0.032333, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(0.085334, 0.036556, 0.0#, 0.049658, -0.048341, 0.0#)

        'Select the centerline and one of the lines
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", -0.00022223294397278, 0.0223334410869282, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0.113556103437018, -0.00144445141358242, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)

        'Suppress the dimension dialog box
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swInputDimValOnCreate, False)

        'Create a full symmetrical angular dimension
        status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", -0.00022223294397278, 0.0223334410869282, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0.113556103437018, -0.00144445141358242, 0, True, 0, Nothing, 0)
        swDisplayDimension = swModelDocExt.AddSymmetricDimension(0.0832913738352659, 0.112403597688285, 0)
        swModel.ClearSelection2(True)

        'Unsuppress the dimension dialog box
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swInputDimValOnCreate, True)

        swModel.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Gear Mate Example (C#)

This example shows how to create a gear mechanical mate.

```
// -----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\MechanicalMates\spurgear.sldasm.
// 2. Delete GearMate1 from the Mates folder in the FeatureManager design tree.
//
// Postconditions: Inspect the Mates folder in the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateGearMate
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 Part;
        private AssemblyDoc swAssembly;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)Part;

            boolstatus = Part.Extension.SelectByRay(-0.00560801689982782, -0.00324062886681986, -0.00252602902980925, -0.57738154511);
            boolstatus = Part.Extension.SelectByRay(0.0367362652137331, -0.000655599730123413, -0.00290761848191323, -0.57738154511);

            // Create GearMateFeatureData
            GearMateFeatureData MateData = (GearMateFeatureData)swAssembly.CreateMateData(10);

            // Set the entities To mate
            object[] EntitiesToMate = new object[2];
            EntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            EntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
            object EntitiesToMateVar = EntitiesToMate;
            MateData.EntitiesToMate = (EntitiesToMateVar);

            // Set the gear ratio numerator
            MateData.GearRatioNumerator = 0.012954;

            // Set the gear ratio denominator
            MateData.GearRatioDenominator = 0.012954;

            // Set the mate orientation direction
            MateData.Reverse = false;

            // Create the mate
            Feature MateFeature = (Feature)swAssembly.CreateMate(MateData);
            Part.ClearSelection2(true);
            Part.EditRebuild3();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create General Table Annotation for SOLIDWORKS MBD 3D PDF Example (C#)

This example shows how to create a general table annotation for SOLIDWORKS MBD 3D PDF.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a general table annotation for SOLIDWORKS MBD 3D PDF.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetGeneralTableAnno_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        TableAnnotation swTable;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swTable = swModelDocExt.GetGeneralTableAnnotation(true, 0, 0, (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "", 2, 2);
            if (swTable == null)
            {
                Debug.Print("Failed to create general table annotation!");
                return;
            }
            else
            {
                Debug.Print("Succeeded in creating general table annotation!");
            }
            Debug.Print(" Total number of rows: " + swTable.TotalRowCount);
            Debug.Print(" Total number of columns: " + swTable.TotalColumnCount);
        }

        public SldWorks swApp;
    }
}
```

Create General Table Annotation for SOLIDWORKS MBD 3D PDF Example (VBA)

This example shows how to create a general table annotation for SOLIDWORKS MBD 3D PDF.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Creates a general table annotation for SOLIDWORKS MBD 3D PDF.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swTable As SldWorks.TableAnnotation  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swTable = swModelDocExt.GetGeneralTableAnnotation(True, 0, 0, swBOMConfigurationAnchor_TopLeft, "", 2, 2)  
  
    Set swTable = swModelDocExt.GetGeneralTableAnnotation(True, 0, 0, swBOMConfigurationAnchor_TopLeft, "", 2, 2)  
    If (swTable Is Nothing) Then  
        Debug.Print "Failed to create general table annotation!"  
        Exit Sub  
    Else  
        Debug.Print "Succeeded in creating general table annotation!"  
    End If  
  
    Debug.Print " Total number of rows: " & swTable.TotalRowCount  
    Debug.Print " Total number of columns: " & swTable.TotalColumnCount  
  
End Sub
```

Create General Table Annotation for SOLIDWORKS MBD 3D PDF Example (VB.NET)

This example shows how to create a general table annotation for SOLIDWORKS MBD 3D PDF.

```
-----
Preconditions:
1. Open a model document.
2. Open the Immediate window.
Postconditions:
1. Creates a general table annotation for SOLIDWORKS MBD 3D PDF.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swTable As TableAnnotation

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swTable = swModelDocExt.GetGeneralTableAnnotation(True, 0, 0, swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, "", 2, 2)
        If (swTable Is Nothing) Then
            Debug.Print("Failed to create general table annotation!")
            Exit Sub
        Else
            Debug.Print("Succeeded in creating general table annotation!")
        End If
        Debug.Print(" Total number of rows: " & swTable.TotalRowCount)
        Debug.Print(" Total number of columns: " & swTable.TotalColumnCount)

    End Sub

    Public swApp As SldWorks

End Class
```

Create Ground Plane Example (C#)

This example shows how to create a ground plane.

```
//-----
// Preconditions: Ensure that the specified model exists.
//
// Postconditions: Creates Ground Plane1 in the Grond Planes folder of the
// FeatureManager design tree And modifies it.
//
// NOTE: Because the model is used elsewhere, do not save any changes to it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateGroundPlane_CSharp
{
    partial class SolidWorksMacro
    {
        private FeatureManager featmgr;
        private GroundPlaneFeatureData swGPData;
        private GroundPlaneFeatureData featdata;
        private Face2 ent;
        private Feature feat;
        private ModelDoc2 Part;
        private AssemblyDoc swAssembly;
        private bool boolstatus;
        private int longstatus = 0;
        private int longwarnings = 0;

        public void Main()
        {
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\assem2.sldasm");

            swAssembly = (AssemblyDoc)Part;
            swApp.ActivateDoc2("assem2.sldasm", false, longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;
            featmgr = Part.FeatureManager;

            boolstatus = Part.Extension.SelectByRay(-0.124913770805165, 0.118854842937424, 0.0702747343940473, -0.620586476260429, -
            ent = (Face2)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            swGPData = (GroundPlaneFeatureData)featmgr.CreateDefinition((int)swFeatureNameID_e.swFmGroundPlane);
            swGPData.PlanarEntity = ent;

            feat = featmgr.CreateFeature(swGPData);
            Part.ClearSelection2(true);
            featdata = (GroundPlaneFeatureData)feat.GetDefinition();
            boolstatus = Part.Extension.SelectByRay(0.119119826446138, 0.00516103810082313, 0.183494239718641, -0.620586476260429, -
            ent = (Face2)((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            featdata.AccessSelections(Part, null);
            featdata.PlanarEntity = ent;
            feat.ModifyDefinition(featdata, Part, null);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create GTol Composite Frame Example (C#)

This example shows how to create a composite frame for a geometric tolerance symbol.

```
//-----
// Preconditions:
// 1. Open a document with a geometric tolerance (GTol) whose first two
//    frames have the same symbol.
// 2. Select the GTol in the graphics area.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Creates a composite frame for the symbol in the graphics area.
// 2. Examine the graphics area and Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetSetCompositeFrame_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        SelectionMgr selMgr;
        ModelDoc2 Part;
        GTol gtol;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            selMgr = (SelectionMgr)Part.SelectionManager;
            gtol = (GTol)selMgr.GetSelectedObject6(1, -1);

            gtol.SetCompositeFrame2(true, 1);
            Debug.Print("Frame 1 is part of a GTol composite frame? " + gtol.GetCompositeFrame2(1));
        }

        public SldWorks swApp;
    }
}
```

Create GTol Composite Frame Example (VBA)

This example shows how to create a composite frame for a geometric tolerance symbol.

```
'-----  
' Preconditions:  
' 1. Open a document with a geometric tolerance (GTol) whose first two  
'    frames have the same symbol.  
' 2. Select the GTol in the graphics area.  
' 3. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Creates a composite frame for the symbol in the graphics area.  
' 2. Examine the graphics area and Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim selMgr As SldWorks.SelectionMgr  
Dim Part As SldWorks.ModelDoc2  
Dim gtol As SldWorks.gtol  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    Set Part = swApp.ActiveDoc  
    Set selMgr = Part.SelectionManager  
    Set gtol = selMgr.GetSelectedObject6(1, -1)  
  
    gtol.SetCompositeFrame2 True, 1  
    Debug.Print "Frame 1 is part of a GTol composite frame? " & gtol.GetCompositeFrame2(1)  
  
End Sub
```

Create GTol Composite Frame Example (VB.NET)

This example shows how to create a composite frame for a geometric tolerance symbol.

```
'-----  
' Preconditions:  
' 1. Open a document with a geometric tolerance (GTol) whose first two  
'    frames have the same symbol.  
' 2. Select the GTol in the graphics area.  
' 3. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Creates a composite frame for the symbol in the graphics area.  
' 2. Examine the graphics area and Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim selMgr As SelectionMgr  
    Dim Part As ModelDoc2  
    Dim gtol As Gtol
```

```
    Sub main()
```

```
        Part = swApp.ActiveDoc  
        selMgr = Part.SelectionManager  
        gtol = selMgr.GetSelectedObject6(1, -1)
```

```
        gtol.SetCompositeFrame2(True, 1)  
        Debug.Print("Frame 1 is part of a GTol composite frame? " & gtol.GetCompositeFrame2(1))
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

```
// -----  
// Preconditions: Open  
// public_documents\samples\tutorial\api\MechanicalMates\Hinge-Assy.sldasm.  
//  
// Postconditions: Inspect the Mates folder in the FeatureManager design tree.  
//  
// NOTE: Because the model is used elsewhere, do not save changes.  
// -----
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateHingeMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 Part;
        private AssemblyDoc swAssembly;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)Part;

            boolstatus = Part.Extension.SelectByRay(0.0343240086878041, 0.09999999999998522, -0.00512270896450673, -0.577381545199981);
            boolstatus = Part.Extension.SelectByRay(0.0824118572918451, 0.100000000000008, -0.0468924659023173, -0.577381545199981);
            boolstatus = Part.Extension.SelectByRay(0.0182257333177063, 0.0419666144006783, 0, -0.577381545199981);
            boolstatus = Part.Extension.SelectByRay(0.0779243258261317, 0.0497185607756023, -0.0384484600391488, -0.577381545199981);
            boolstatus = Part.Extension.SelectByRay(0.0533819856866558, 0.0921914939762019, -0.00131731445623018, -0.577381545199981);
            boolstatus = Part.Extension.SelectByRay(0.0580246157638271, 0.0700560308736158, -0.0192219802344198, -0.577381545199981);

            // Create HingeMateFeatureData
            HingeMateFeatureData MateData = (HingeMateFeatureData)swAssembly.CreateMateData(22);

            // Set the concentric entities to mate
            object[] ConcentricEntitiesToMate = new object[2];
            ConcentricEntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(5, -1);
            ConcentricEntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(6, -1);
            object ConcentricEntitiesToMateVar = ConcentricEntitiesToMate;
            MateData.EntitiesToMate[0] = ConcentricEntitiesToMateVar;

            // Set the coincident entities to mate
            object[] CoincidentEntitiesToMate = new object[2];
            CoincidentEntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(1, -1);
            CoincidentEntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(2, -1);
            object CoincidentEntitiesToMateVar = CoincidentEntitiesToMate;
            MateData.EntitiesToMate[1] = CoincidentEntitiesToMateVar;

            // Set the mate angle selection
            MateData.AngleSelection = true;

            // Set the angle entities to mate
            object[] AngleEntitiesToMate = new object[2];
            AngleEntitiesToMate[0] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(3, -1);
            AngleEntitiesToMate[1] = ((SelectionMgr)(Part.SelectionManager)).GetSelectedObject6(4, -1);
            object AngleEntitiesToMateVar = AngleEntitiesToMate;
            MateData.EntitiesToMate[2] = AngleEntitiesToMateVar;

            // Set the mate angle value
            MateData.Angle = 0.5235987755983;

            // Set the mate angle minimum value
            MateData.MinVal = 0.5235987755983;

            // Set the mate angle maximum value
            MateData.MaxVal = 0.5235987755983;

            // Set the mate alignment
            MateData.MateAlignment = 0;

            // Create the mate
            Feature MateFeature = (Feature)swAssembly.CreateMate(MateData);
            Part.ClearSelection2(true);
            Part.EditRebuild3();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
```

```
    /// </summary>  
    public Sldworks swApp;  
}
```

Create Hole Wizard Feature Data Object and Hole Wizard Feature Example (VBA)

This example shows how to create a hole wizard feature data object and a hole wizard feature.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\block20.sldprt.
'
' Postconditions:
' 1. Creates a hole wizard feature on the selected face of the part.
' 2. Examine the FeatureManager design tree and graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swWzdHole As WizardHoleFeatureData2
Dim swFeatDataObj As Object
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swFeatMgr = swModel.FeatureManager

    ' Create the hole wizard feature data object
    Set swFeatDataObj = swFeatMgr.CreateDefinition(swFmHoleWzd)
    Set swWzdHole = swFeatDataObj

    ' Initialize the hole wizard feature
    swWzdHole.InitializeHole swWzdCounterBore, swStandardISO, swStandardISOHexBolt, "M48", swEndCondBlind

    'Change the hole wizard feature's depth
    swWzdHole.HoleDepth = 0.15

    'Change the hole wizard feature's diameter
    swWzdHole.HoleDiameter = 0.0555

    ' Select the face on which to create the hole
    boolstatus = swModelDocExt.SelectByID("", "FACE", 9.30621655732011E-03, 3.96239999998897E-02, -3.32715966641217E-03, False, 0, Nothing, 0)

    ' Create the hole wizard feature
    Set swFeat = swFeatMgr.CreateFeature(swWzdHole)

End Sub
```

Create Hole Wizard Hole Example (C#)

This example shows how to create a hole using the hole wizard.

```
//-----
// Preconditions: Verify that the part template exists.
//
// Postconditions:
// 1. Creates a part.
// 2. Inserts a hole in the part using the hole wizard.
// 3. Examine the graphics area and FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            SketchManager swSketchMgr = default(SketchManager);
            object sketchLines = null;
            int status = 0;
            bool boolstatus = false;
            double[] P1 = new double[3];
            double[] P2 = new double[3];
            double[] P3 = new double[3];

            //Create the model for the wizard hole
            swApp.ResetUntitledCount(0, 0, 0);
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2020\\templates\\Part.prtdot", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref status);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSketchMgr = swModel.SketchManager;
            swModelDocExt = swModel.Extension;
            swFeatMgr = swModel.FeatureManager;
            sketchLines = swSketchMgr.CreateCornerRectangle(-0.05096498314664, 0.05060941349678, 0, 0.1021670127265, -0.05037236706354, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            swFeat = swFeatMgr.FeatureExtrusion2(true, false, false, false, 0, 0.381, 0.381, false, false, false, false, 0.01745329251994, 0.01745329251994,
            false, false, false, false, true, true, true, 0, 0, false);

            //Create three points for the reference plane
            P1[0] = -0.0141556764402858;
            P1[1] = 0.00194061273859598;
            P1[2] = 0;
            P2[0] = -0.0141556764402858;
            P2[1] = 0.00194061273859598;
            P2[2] = 1;
            P3[0] = -0.149976101832345;
            P3[1] = -0.988792859011662;
            P3[2] = 0;

            //Create the reference plane
            swModel.CreatePlaneFixed2(P1, P2, P3, false);

            //Select reference plane
            boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", -0.0156784487003801, -0.00916715285390111, 0.0558270998665543, false, 0, null, 0);

            //Create the hole wizard countersink hole
            swFeat = swFeatMgr.HoleWizard5((int)swWzdGeneralHoleTypes_e.swWzdCounterSink, (int)swWzdHoleStandards_e.swStandardAnsiMetric,
            (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiMetricFlatHead82, "M2", (int)swEndConditions_e.swEndCondThroughAll, 0.0102, 0.010312189893273, 0,
            0.0044, 1.57079632679489, 0.000152189893272978, 0, -1, -1, -1, -1, 1, -1, -1, -1, "", false, true, true, true, true, false);
        }

        public SldWorks swApp;
    }
}
```

Create Hole Wizard Hole Example (VBA)

This example shows how to create a hole using the hole wizard.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates a part.
' 2. Creates a hole using the hole wizard.
' 3. Examine the graphics area and FeatureManager
'    design tree.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swSketchMgr As SldWorks.SketchManager
Dim sketchLines As Variant
Dim longstatus As Long
Dim boolstatus As Boolean
Dim P1(2) As Double
Dim P2(2) As Double
Dim P3(2) As Double

Sub main()

    Set swApp = Application.SldWorks

    'Create the part for the wizard hole
    swApp.ResetUntitledCount 0, 0, 0
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2020\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set swModel = swApp.ActiveDoc
    Set swSketchMgr = swModel.SketchManager
    Set swModelDocExt = swModel.Extension
    Set swFeatMgr = swModel.FeatureManager
    sketchLines = swSketchMgr.CreateCornerRectangle(-0.05096498314664, 0.05060941349678, 0, 0.1021670127265, -0.05037236706354, 0)
    boolstatus = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeat = swFeatMgr.FeatureExtrusion2(True, False, False, 0, 0, 0.381, 0.381, False, False, False, False, 0.01745329251994, 0.01745329251994,
    False, False, False, False, True, True, True, 0, 0, False)

    'Create three points for the reference plane
    P1(0) = -1.41556764402858E-02
    P1(1) = 1.94061273859598E-03
    P1(2) = 0
    P2(0) = -1.41556764402858E-02
    P2(1) = 1.94061273859598E-03
    P2(2) = 1
    P3(0) = -0.149976101832345
    P3(1) = -0.988792859011662
    P3(2) = 0

    'Create the reference plane
    swModel.CreatePlaneFixed2 P1, P2, P3, False

    'Select reference plane
    boolstatus = swModelDocExt.SelectByID("Plane1", "PLANE", -1.56784487003801E-02, -9.16715285390111E-03, 5.58270998665543E-02, False, 0, Nothing, 0)

    'Create the hole wizard countersink hole
    Set swFeat = swFeatMgr.HoleWizard5(swWzdCounterSink, swStandardAnsiMetric, swStandardAnsiMetricFlatHead82, "M2", swEndCondThroughAll, 0.0102,
    0.010312189893273, 0, 0.0044, 1.57079632679489, 1.52189893272978E-04, 0, -1, -1, -1, -1, 1, -1, -1, -1, "", False, True, True, True, True, False)

End Sub
```

Create Hole Wizard Hole Example (VB.NET)

This example shows how to create a hole using the hole wizard.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates a part.
' 2. Creates a hole using the hole wizard.
' 3. Examine the graphics area and FeatureManager
'    design tree.
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Partial Class SolidWorksMacro

```
    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatMgr As FeatureManager
        Dim swFeat As Feature
        Dim swSketchMgr As SketchManager
        Dim sketchLines As Object
        Dim status As Integer
        Dim boolstatus As Boolean
        Dim P1(2) As Double
        Dim P2(2) As Double
        Dim P3(2) As Double

        'Create the model for the wizard hole
        swApp.ResetUntitledCount(0, 0, 0)
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2020\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, status)
        swModel = swApp.ActiveDoc
        swSketchMgr = swModel.SketchManager
        swModelDocExt = swModel.Extension
        swFeatMgr = swModel.FeatureManager
        sketchLines = swSketchMgr.CreateCornerRectangle(-0.050996498314664, 0.05060941349678, 0, 0.1021670127265, -0.05037236706354, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        swFeat = swFeatMgr.FeatureExtrusion2(True, False, False, 0, 0, 0.381, 0.381, False, False, False, False, 0.01745329251994, 0.01745329251994, False
, False, False, False, True, True, 0, 0, False)

        'Create three points for the reference plane
        P1(0) = -0.0141556764402858
        P1(1) = 0.00194061273859598
        P1(2) = 0
        P2(0) = -0.0141556764402858
        P2(1) = 0.00194061273859598
        P2(2) = 1
        P3(0) = -0.149976101832345
        P3(1) = -0.988792859011662
        P3(2) = 0

        'Create the reference plane
        swModel.CreatePlaneFixed2(P1, P2, P3, False)

        'Select the reference plane
        boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", -0.0156784487003801, -0.00916715285390111, 0.0558270998665543, False, 0, Nothing, 0)

        'Create the hole wizard countersink hole
        swFeat = swFeatMgr.HoleWizard5(swWzdGeneralHoleTypes_e.swWzdCounterSink, swWzdHoleStandards_e.swStandardAnsiMetric, swWzdHoleStandardFastenerTypes
_e.swStandardAnsiMetricFlatHead82, "M2", swEndConditions_e.swEndCondThroughAll, 0.0102, 0.010312189893273, 0, 0.0044, 1.57079632679489, 0.0001521898932729
78, 0, -1, -1, -1, 1, -1, -1, -1, "", False, True, True, True, True, False)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
```

End Class

Create Holes Using Hole Wizard and Sketch Points Example (C#)

This example shows how to create holes using the Hole Wizard and sketch points.

```
//-----
// Problem:
// When adding a number of Hole Wizard hole features,
// it is often helpful to use a sketch with a number of
// sketch points. However, IFeatureManager::HoleWizard
// requires selection points to be on a face and in the
// space of the model.
//
// This example shows how to transform a sketch point
// from the space of a sketch to the space of the model.
//
// Preconditions:
// 1. Open a part or assembly document.
// 2. Fully resolve the assembly.
// 3. Select a sketch with at least one sketch point.
//
// Postconditions:
// 1. Inserts counterbore hole features at each sketch point.
// 2. Inspect the Immediate window.
//
// NOTES:
// 1. Hole creation can fail due to geometry conditions.
// 2. To access:
//    * IWizardHoleFeatureData2::Face
//    * IWizardHoleFeatureData2::Vertex
//    * IWizardHoleFeatureData2::GetEndConditionReference
// the feature must be rolled back with a call to
// IWizardHoleFeatureData2::AccessSelections
// because these are geometric entities that
// can be consumed by subsequent features.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace WizardHoleFeatureData_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public bool CreateHoleWizardFeature(SldWorks swApp, ModelDoc2 swModel)
        {
            bool functionReturnValue = false;

            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            WizardHoleFeatureData2 swWizHole = default(WizardHoleFeatureData2);

            swFeatMgr = swModel.FeatureManager;
            swFeat = swFeatMgr.HoleWizard((int)swWzdGeneralHoleTypes_e.swWzdCounterBore, (int)swWzdHoleStandards_e.swStandardISO,
            (int)swWzdHoleStandardFastenerTypes_e.swStandardISOHexCapScrew, "M6", (int)swWzdHoleThreadEndCondition_e.swEndThreadTypeBLIND,
            0.0066, 0.02, 0.014547, 0.004, 0.0,
            1.0, 2.059488517353, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0);

            // Hole creation can fail due to geometry conditions
            if (swFeat == null)
            {
                functionReturnValue = false;
                return functionReturnValue;
            }

            swWizHole = (WizardHoleFeatureData2)swFeat.GetDefinition();

            Debug.Print("Feature = " + swFeat.Name);
            Debug.Print("    Type                                = " + swWizHole.Type);
            Debug.Print("");
            Debug.Print("    CosmeticThreadType                = " + swWizHole.CosmeticThreadType);
            Debug.Print("    CounterBoreDepth                  = " + swWizHole.CounterBoreDepth * 1000.0 + " mm");
            Debug.Print("    CounterBoreDiameter                = " + swWizHole.CounterBoreDiameter * 1000.0 + " mm");
            //1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print("    CounterDrillAngle                  = " + swWizHole.CounterDrillAngle * 57.3 + " deg");
            Debug.Print("    CounterDrillDepth                  = " + swWizHole.CounterDrillDepth * 1000.0 + " mm");
            Debug.Print("    CounterDrillDiameter                = " + swWizHole.CounterDrillDiameter * 1000.0 + " mm");
            Debug.Print("    CounterSinkAngle                  = " + swWizHole.CounterSinkAngle * 57.3 + " deg");
            Debug.Print("    CounterSinkDiameter                = " + swWizHole.CounterSinkDiameter * 1000.0 + " mm");
            Debug.Print("    Depth                              = " + swWizHole.Depth * 1000.0 + " mm");
            Debug.Print("    Diameter                          = " + swWizHole.Diameter * 1000.0 + " mm");
            Debug.Print("    DrillAngle                        = " + swWizHole.DrillAngle * 57.3 + " deg");
            Debug.Print("    EndCondition                      = " + swWizHole.EndCondition);
        }
    }
}
```

```

Debug.Print(" FarCounterSinkAngle" = " + swWizHole.FarCounterSinkAngle * 57.3 + " deg");
Debug.Print(" FarCounterSinkDiameter" = " + swWizHole.FarCounterSinkDiameter * 1000.0 + " mm");
Debug.Print(" FastenerSize" = " + swWizHole.FastenerSize);
Debug.Print(" FastenerType" = " + swWizHole.FastenerType2);
Debug.Print(" HeadClearance" = " + swWizHole.HeadClearance * 1000.0 + " mm");
Debug.Print(" HeadClearanceType" = " + swWizHole.HeadClearanceType);
Debug.Print(" HoleDepth" = " + swWizHole.HoleDepth * 1000.0 + " mm");
Debug.Print(" HoleDiameter" = " + swWizHole.HoleDiameter * 1000.0 + " mm");
Debug.Print(" MajorDiameter" = " + swWizHole.MajorDiameter * 1000.0 + " mm");
Debug.Print(" MidCounterSinkAngle" = " + swWizHole.MidCounterSinkAngle * 57.3 + " deg");
Debug.Print(" MidCounterSinkDiameter" = " + swWizHole.MidCounterSinkDiameter * 1000.0 + " mm");
Debug.Print(" MinorDiameter" = " + swWizHole.MinorDiameter * 1000.0 + " mm");
Debug.Print(" NearCounterSinkAngle" = " + swWizHole.NearCounterSinkAngle * 57.3 + " deg");
Debug.Print(" NearCounterSinkDiameter" = " + swWizHole.NearCounterSinkDiameter * 1000.0 + " mm");
Debug.Print(" Standard" = " + swWizHole.Standard2);
Debug.Print(" TapDrillDepth" = " + swWizHole.TapDrillDepth * 1000.0 + " mm");
Debug.Print(" TapDrillDiameter" = " + swWizHole.TapDrillDiameter * 1000.0 + " mm");
Debug.Print(" ThreadAngle" = " + swWizHole.ThreadAngle * 57.3 + " deg");
Debug.Print(" ThreadDepth" = " + swWizHole.ThreadDepth * 1000.0 + " mm");
Debug.Print(" ThreadDiameter" = " + swWizHole.ThreadDiameter * 1000.0 + " mm");
Debug.Print(" ThruHoleDepth" = " + swWizHole.ThruHoleDepth * 1000.0 + " mm");
Debug.Print(" ThruHoleDiameter" = " + swWizHole.ThruHoleDiameter * 1000.0 + " mm");
Debug.Print(" ThruTapDrillDepth" = " + swWizHole.ThruTapDrillDepth * 1000.0 + " mm");
Debug.Print(" ThruTapDrillDiameter" = " + swWizHole.ThruTapDrillDiameter * 1000.0 + " mm");

functionReturnValue = true;
return functionReturnValue;
}

public void Main()
{
    MathUtility swMathUtil = default(MathUtility);
    ModelDoc2 swModel = default(ModelDoc2);
    SelectionMgr swSelMgr = default(SelectionMgr);
    Feature swFeat = default(Feature);
    Sketch swSketch = default(Sketch);
    object[] vSketchPtArr = null;
    object vSketchPt = null;
    SketchPoint swSketchPt = default(SketchPoint);
    MathTransform swSketchXform = default(MathTransform);
    object vPtArr = null;
    MathPoint swPt = default(MathPoint);
    bool bRet = false;
    double[] nPtData = new double[3];

    swMathUtil = (MathUtility)swApp.GetMathUtility();
    swModel = (ModelDoc2)swApp.ActiveDoc;
    swSelMgr = (SelectionMgr)swModel.SelectionManager;
    swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
    swSketch = (Sketch)swFeat.GetSpecificFeature2();
    swSketchXform = swSketch.ModelToSketchTransform;
    swSketchXform = (MathTransform)swSketchXform.Inverse();

    vSketchPtArr = (object[])swSketch.GetSketchPoints();

    foreach (object vSketchPt_loopVariable in vSketchPtArr)
    {
        vSketchPt = vSketchPt_loopVariable;
        swSketchPt = (SketchPoint)vSketchPt;
        nPtData[0] = swSketchPt.X;
        nPtData[1] = swSketchPt.Y;
        nPtData[2] = swSketchPt.Z;
        vPtArr = nPtData;

        swPt = (MathPoint)swMathUtil.CreatePoint(vPtArr);
        swPt = (MathPoint)swPt.MultiplyTransform(swSketchXform);

        bRet = swModel.Extension.SelectByID2("", "FACE", ((double[])swPt.ArrayData)[0], ((double[])swPt.ArrayData)[1],
        ((double[])swPt.ArrayData)[2], true, 0, null, 0);
        bRet = CreateHoleWizardFeature(swApp, swModel);
    }
}

public SldWorks swApp;
}
}

```

Create Holes Using Hole Wizard and Sketch Points Example (VBA)

This example shows how to create holes using the Hole Wizard and sketch points.

```

'-----
' Problem:
' When adding a number of Hole Wizard hole features,
' it is often helpful to use a sketch with a number of
' sketch points. However, IFeatureManager::HoleWizard
' requires selection points to be on a face and in the
' space of the model.
'
' This example shows how to transform a sketch point
' from the space of a sketch to the space of the model.
'
' Preconditions:
' 1. Open a part or assembly document.
' 2. Fully resolve the assembly.
' 3. Select a sketch with at least one sketch point.
'
' Postconditions:
' 1. Inserts counterbore hole features at each sketch point.
' 2. Inspect the Immediate window.
'
' NOTES:
' 1. Hole creation can fail due to geometry conditions.
' 2. To access:
'    * IWizardHoleFeatureData2::Face
'    * IWizardHoleFeatureData2::Vertex
'    * IWizardHoleFeatureData2::GetEndConditionReference
'    the feature must be rolled back with a call to
'    IWizardHoleFeatureData2::AccessSelections
'    because these are geometric entities that
'    can be consumed by subsequent features.
'-----

```

Option Explicit

```
Function CreateHoleWizardFeature(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2) As Boolean
```

```
    Dim swFeatMgr          As SldWorks.FeatureManager
    Dim swFeat              As SldWorks.Feature
    Dim swWizHole           As SldWorks.WizardHoleFeatureData2
```

```
    Set swFeatMgr = swModel.FeatureManager
    Set swFeat = swFeatMgr.HoleWizard _
```

```
        ( _
        swWzdCounterBore, _
        swStandardISO, _
        swStandardISOHexCapScrew, _
        "M6", _
        swEndThreadTypeBLIND, _
        0.0066, _
        0.02, _
        0.014547, _
        0.004, _
        0#, _
        1#, _
        2.059488517353, _
        0#, _
        0#, _
        0#, _
        0#, _
        0#, _
        0#, _
        0# _
        )
```

```
    ' Hole creation can fail due to geometry conditions
```

```
    If swFeat Is Nothing Then
        CreateHoleWizardFeature = False
        Exit Function
    End If
```

```
    Set swWizHole = swFeat.GetDefinition
```

```
    Debug.Print "Feature = " + swFeat.Name
    Debug.Print "    Type                = " & swWizHole.Type
    Debug.Print ""
    Debug.Print "    CosmeticThreadType            = " & swWizHole.CosmeticThreadType
    Debug.Print "    CounterBoreDepth              = " & swWizHole.CounterBoreDepth * 1000# & " mm"
    Debug.Print "    CounterBoreDiameter           = " & swWizHole.CounterBoreDiameter * 1000# & " mm"
    '1 radian = 180°/p = 57.295779513° or approximately 57.3°
    Debug.Print "    CounterDrillAngle              = " & swWizHole.CounterDrillAngle * 57.3 & " deg"
    Debug.Print "    CounterDrillDepth              = " & swWizHole.CounterDrillDepth * 1000# & " mm"
    Debug.Print "    CounterDrillDiameter           = " & swWizHole.CounterDrillDiameter * 1000# & " mm"
    Debug.Print "    CounterSinkAngle               = " & swWizHole.CounterSinkAngle * 57.3 & " deg"
    Debug.Print "    CounterSinkDiameter            = " & swWizHole.CounterSinkDiameter * 1000# & " mm"
    Debug.Print "    Depth                          = " & swWizHole.Depth * 1000# & " mm"
    Debug.Print "    Diameter                       = " & swWizHole.Diameter * 1000# & " mm"
    Debug.Print "    DrillAngle                     = " & swWizHole.DrillAngle * 57.3 & " deg"
```

```

Debug.Print " EndCondition = " & swWizHole.EndCondition
Debug.Print " FarCounterSinkAngle = " & swWizHole.FarCounterSinkAngle * 57.3 & " deg"
Debug.Print " FarCounterSinkDiameter = " & swWizHole.FarCounterSinkDiameter * 1000# & " mm"
Debug.Print " FastenerSize = " & swWizHole.FastenerSize
Debug.Print " FastenerType = " & swWizHole.FastenerType2
Debug.Print " HeadClearance = " & swWizHole.HeadClearance * 1000# & " mm"
Debug.Print " HeadClearanceType = " & swWizHole.HeadClearanceType
Debug.Print " HoleDepth = " & swWizHole.HoleDepth * 1000# & " mm"
Debug.Print " HoleDiameter = " & swWizHole.HoleDiameter * 1000# & " mm"
Debug.Print " MajorDiameter = " & swWizHole.MajorDiameter * 1000# & " mm"
Debug.Print " MidCounterSinkAngle = " & swWizHole.MidCounterSinkAngle * 57.3 & " deg"
Debug.Print " MidCounterSinkDiameter = " & swWizHole.MidCounterSinkDiameter * 1000# & " mm"
Debug.Print " MinorDiameter = " & swWizHole.MinorDiameter * 1000# & " mm"
Debug.Print " NearCounterSinkAngle = " & swWizHole.NearCounterSinkAngle * 57.3 & " deg"
Debug.Print " NearCounterSinkDiameter = " & swWizHole.NearCounterSinkDiameter * 1000# & " mm"
Debug.Print " Standard = " & swWizHole.Standard2
Debug.Print " TapDrillDepth = " & swWizHole.TapDrillDepth * 1000# & " mm"
Debug.Print " TapDrillDiameter = " & swWizHole.TapDrillDiameter * 1000# & " mm"
Debug.Print " ThreadAngle = " & swWizHole.ThreadAngle * 57.3 & " deg"
Debug.Print " ThreadDepth = " & swWizHole.ThreadDepth * 1000# & " mm"
Debug.Print " ThreadDiameter = " & swWizHole.ThreadDiameter * 1000# & " mm"
Debug.Print " ThruHoleDepth = " & swWizHole.ThruHoleDepth * 1000# & " mm"
Debug.Print " ThruHoleDiameter = " & swWizHole.ThruHoleDiameter * 1000# & " mm"
Debug.Print " ThruTapDrillDepth = " & swWizHole.ThruTapDrillDepth * 1000# & " mm"
Debug.Print " ThruTapDrillDiameter = " & swWizHole.ThruTapDrillDiameter * 1000# & " mm"

```

```
CreateHoleWizardFeature = True
```

```
End Function
```

```
Sub main()
```

```

Dim swApp As SldWorks.SldWorks
Dim swMathUtil As SldWorks.MathUtility
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim swSketch As SldWorks.Sketch
Dim vSketchPtArr As Variant
Dim vSketchPt As Variant
Dim swSketchPt As SldWorks.SketchPoint
Dim swSketchXform As SldWorks.MathTransform
Dim vPtArr As Variant
Dim swPt As SldWorks.MathPoint
Dim bRet As Boolean
Dim nPtData(2) As Double

```

```

Set swApp = Application.SldWorks
Set swMathUtil = swApp.GetMathUtility
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
Set swSketch = swFeat.GetSpecificFeature2
Set swSketchXform = swSketch.ModelToSketchTransform
Set swSketchXform = swSketchXform.Inverse

```

```
vSketchPtArr = swSketch.GetSketchPoints
```

```
For Each vSketchPt In vSketchPtArr
```

```

Set swSketchPt = vSketchPt
nPtData(0) = swSketchPt.X
nPtData(1) = swSketchPt.Y
nPtData(2) = swSketchPt.Z
vPtArr = nPtData

```

```

Set swPt = swMathUtil.CreatePoint(vPtArr)
Set swPt = swPt.MultiplyTransform(swSketchXform)

```

```
bRet = swModel.Extension.SelectByID2("", "FACE", swPt.ArrayData(0), swPt.ArrayData(1), swPt.ArrayData(2), True, 0, Nothing,
```

```
0)
```

```
bRet = CreateHoleWizardFeature(swApp, swModel)
```

```
Next
```

```
End Sub
```

Create Holes Using Hole Wizard and Sketch Points Example (VB.NET)

This example shows how to create holes using the Hole Wizard and sketch points.

```

'-----
' Problem:
' When adding a number of Hole Wizard hole features,
' it is often helpful to use a sketch with a number of
' sketch points. However, IFeatureManager::HoleWizard'
' requires selection points to be on a face and in the
' space of the model.
'
' This example shows how to transform a sketch point
' from the space of a sketch to the space of the model.
'
' Preconditions:
' 1. Open a part or assembly document.
' 2. Fully resolve the assembly.
' 3. Select a sketch with at least one sketch point.
'
' Postconditions:
' 1. Inserts counterbore hole features at each sketch point.
' 2. Inspect the Immediate window.
'
' NOTES:
' 1. Hole creation can fail due to geometry conditions.
' 2. To access:
'    * IWizardHoleFeatureData2::Face
'    * IWizardHoleFeatureData2::Vertex
'    * IWizardHoleFeatureData2::GetEndConditionReference
' the feature must be rolled back with a call to
' IWizardHoleFeatureData2::AccessSelections
' because these are geometric entities that
' can be consumed by subsequent features.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

Function CreateHoleWizardFeature(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2) As Boolean

```

    Dim swFeatMgr As FeatureManager
    Dim swFeat As Feature
    Dim swWizHole As WizardHoleFeatureData2

```

```

    swFeatMgr = swModel.FeatureManager
    swFeat = swFeatMgr.HoleWizard _

```

```

        ( _
            swWzdGeneralHoleTypes_e.swWzdCounterBore, _
            swWzdHoleStandards_e.swStandardISO, _
            swWzdHoleStandardFastenerTypes_e.swStandardISOHexCapScrew, _
            "M6", _
            swWzdHoleThreadEndCondition_e.swEndThreadTypeBLIND, _
            0.0066, _
            0.02, _
            0.014547, _
            0.004, _
            0.0#, _
            1.0#, _
            2.059488517353, _
            0.0#, _
            0.0#, _
            0.0#, _
            0.0#, _
            0.0#, _
            0.0#, _
            0.0# _
        )

```

```

' Hole creation can fail due to geometry conditions

```

```

If swFeat Is Nothing Then
    CreateHoleWizardFeature = False
    Exit Function
End If

```

```

swWizHole = swFeat.GetDefinition

```

```

Debug.Print("Feature = " + swFeat.Name)
Debug.Print("    Type                                = " & swWizHole.Type)
Debug.Print("")
Debug.Print("    CosmeticThreadType                = " & swWizHole.CosmeticThreadType)
Debug.Print("    CounterBoreDepth                  = " & swWizHole.CounterBoreDepth * 1000.0# & " mm")
Debug.Print("    CounterBoreDiameter                = " & swWizHole.CounterBoreDiameter * 1000.0# & " mm")
'1 radian = 180°/p = 57.295779513° or approximately 57.3°

```

```

Debug.Print(" CounterDrillAngle = " & swWizHole.CounterDrillAngle * 57.3 & " deg")
Debug.Print(" CounterDrillDepth = " & swWizHole.CounterDrillDepth * 1000.0# & " mm")
Debug.Print(" CounterDrillDiameter = " & swWizHole.CounterDrillDiameter * 1000.0# & " mm")
Debug.Print(" CounterSinkAngle = " & swWizHole.CounterSinkAngle * 57.3 & " deg")
Debug.Print(" CounterSinkDiameter = " & swWizHole.CounterSinkDiameter * 1000.0# & " mm")
Debug.Print(" Depth = " & swWizHole.Depth * 1000.0# & " mm")
Debug.Print(" Diameter = " & swWizHole.Diameter * 1000.0# & " mm")
Debug.Print(" DrillAngle = " & swWizHole.DrillAngle * 57.3 & " deg")
Debug.Print(" EndCondition = " & swWizHole.EndCondition)
Debug.Print(" FarCounterSinkAngle = " & swWizHole.FarCounterSinkAngle * 57.3 & " deg")
Debug.Print(" FarCounterSinkDiameter = " & swWizHole.FarCounterSinkDiameter * 1000.0# & " mm")
Debug.Print(" FastenerSize = " & swWizHole.FastenerSize)
Debug.Print(" FastenerType = " & swWizHole.FastenerType2)
Debug.Print(" HeadClearance = " & swWizHole.HeadClearance * 1000.0# & " mm")
Debug.Print(" HeadClearanceType = " & swWizHole.HeadClearanceType)
Debug.Print(" HoleDepth = " & swWizHole.HoleDepth * 1000.0# & " mm")
Debug.Print(" HoleDiameter = " & swWizHole.HoleDiameter * 1000.0# & " mm")
Debug.Print(" MajorDiameter = " & swWizHole.MajorDiameter * 1000.0# & " mm")
Debug.Print(" MidCounterSinkAngle = " & swWizHole.MidCounterSinkAngle * 57.3 & " deg")
Debug.Print(" MidCounterSinkDiameter = " & swWizHole.MidCounterSinkDiameter * 1000.0# & " mm")
Debug.Print(" MinorDiameter = " & swWizHole.MinorDiameter * 1000.0# & " mm")
Debug.Print(" NearCounterSinkAngle = " & swWizHole.NearCounterSinkAngle * 57.3 & " deg")
Debug.Print(" NearCounterSinkDiameter = " & swWizHole.NearCounterSinkDiameter * 1000.0# & " mm")
Debug.Print(" Standard = " & swWizHole.Standard2)
Debug.Print(" TapDrillDepth = " & swWizHole.TapDrillDepth * 1000.0# & " mm")
Debug.Print(" TapDrillDiameter = " & swWizHole.TapDrillDiameter * 1000.0# & " mm")
Debug.Print(" ThreadAngle = " & swWizHole.ThreadAngle * 57.3 & " deg")
Debug.Print(" ThreadDepth = " & swWizHole.ThreadDepth * 1000.0# & " mm")
Debug.Print(" ThreadDiameter = " & swWizHole.ThreadDiameter * 1000.0# & " mm")
Debug.Print(" ThruHoleDepth = " & swWizHole.ThruHoleDepth * 1000.0# & " mm")
Debug.Print(" ThruHoleDiameter = " & swWizHole.ThruHoleDiameter * 1000.0# & " mm")
Debug.Print(" ThruTapDrillDepth = " & swWizHole.ThruTapDrillDepth * 1000.0# & " mm")
Debug.Print(" ThruTapDrillDiameter = " & swWizHole.ThruTapDrillDiameter * 1000.0# & " mm")

```

```
CreateHoleWizardFeature = True
```

```
End Function
```

```
Sub main()
```

```

Dim swMathUtil As MathUtility
Dim swModel As ModelDoc2
Dim swSelMgr As SelectionMgr
Dim swFeat As Feature
Dim swSketch As Sketch
Dim vSketchPtArr As Object
Dim vSketchPt As Object
Dim swSketchPt As SketchPoint
Dim swSketchXform As MathTransform
Dim vPtArr As Object
Dim swPt As MathPoint
Dim bRet As Boolean
Dim nPtData(2) As Double

swMathUtil = swApp.GetMathUtility
swModel = swApp.ActiveDoc
swSelMgr = swModel.SelectionManager
swFeat = swSelMgr.GetSelectedObject6(1, -1)
swSketch = swFeat.GetSpecificFeature2
swSketchXform = swSketch.ModelToSketchTransform
swSketchXform = swSketchXform.Inverse

```

```
vSketchPtArr = swSketch.GetSketchPoints
```

```
For Each vSketchPt In vSketchPtArr
```

```

    swSketchPt = vSketchPt
    nPtData(0) = swSketchPt.X
    nPtData(1) = swSketchPt.Y
    nPtData(2) = swSketchPt.Z
    vPtArr = nPtData

    swPt = swMathUtil.CreatePoint(vPtArr)
    swPt = swPt.MultiplyTransform(swSketchXform)

```

```
Nothing, 0)
bRet = swModel.Extension.SelectByID2("", "FACE", swPt.ArrayData(0), swPt.ArrayData(1), swPt.ArrayData(2), True, 0,
```

```
bRet = CreateHoleWizardFeature(swApp, swModel)
```

```
Next
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Imported Solid Body Example (C#)

This example shows how to create an imported solid body in the shape of a pyramid.

```
//-----
// Preconditions: Verify that the specified part
// document template exists.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates a pyramid-shaped, imported, solid body.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace MacroCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            PartDoc swPart = default(PartDoc);
            Body2 swBody = default(Body2);
            double[] nPt = null;
            object vPt = null;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\part.prt", 0, 0, 0);
            swPart = (PartDoc)swModel;
            swBody = (Body2)swPart.CreateNewBody();
            // Front
            nPt = new double[9];
            nPt[0] = 0.0;
            nPt[1] = 0.0;
            nPt[2] = 1.0;
            nPt[3] = -1.0;
            nPt[4] = -1.0;
            nPt[5] = 0.0;
            nPt[6] = 1.0;
            nPt[7] = -1.0;
            nPt[8] = 0.0;
            vPt = nPt;
            bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), null);
            // Left
            nPt = new double[9];
            nPt[0] = 0.0;
            nPt[1] = 0.0;
            nPt[2] = 1.0;
            nPt[3] = -1.0;
            nPt[4] = -1.0;
            nPt[5] = 0.0;
            nPt[6] = -1.0;
            nPt[7] = 1.0;
            nPt[8] = 0.0;
            vPt = nPt;
            bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), null);
            // Back
            nPt = new double[9];
            nPt[0] = 0.0;
            nPt[1] = 0.0;
            nPt[2] = 1.0;
            nPt[3] = -1.0;
            nPt[4] = 1.0;
            nPt[5] = 0.0;
            nPt[6] = 1.0;
            nPt[7] = 1.0;
            nPt[8] = 0.0;
            vPt = nPt;
            bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), null);
            // Right
            nPt = new double[9];
            nPt[0] = 0.0;
            nPt[1] = 0.0;
            nPt[2] = 1.0;
            nPt[3] = 1.0;
            nPt[4] = 1.0;
            nPt[5] = 0.0;
            nPt[6] = 1.0;
            nPt[7] = -1.0;
            nPt[8] = 0.0;
            vPt = nPt;
            bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), null);
            // Bottom
            nPt = new double[12];
            nPt[0] = -1.0;
            nPt[1] = -1.0;
            nPt[2] = 0.0;
            nPt[3] = -1.0;
            nPt[4] = 1.0;
            nPt[5] = 0.0;
            nPt[6] = 1.0;
            nPt[7] = 1.0;
            nPt[8] = 0.0;
            nPt[9] = 1.0;
            nPt[10] = -1.0;
            nPt[11] = 0.0;
            vPt = nPt;
            bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), null);
        }
    }
}
```

```
        bRet = swBody.CreateBodyFromSurfaces();  
        swModel.ViewZoomtofit2();  
    }  
    /// <summary>  
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

Create Imported Solid Body Example (VBA)

This example shows how to create an imported solid body in the shape of a pyramid.

```
'-----
' Preconditions: Verify that the specified part
' document template exists.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a pyramid-shaped, imported, solid body.
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swPart As SldWorks.PartDoc
Dim swBody As SldWorks.Body2
Dim nPt() As Double
Dim vPt As Variant
Dim bRet As Boolean

Set swApp = Application.SldWorks
Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\part.prtdot", 0, 0, 0)
Set swPart = swModel
Set swBody = swPart.CreateNewBody

' Front
ReDim nPt(8)
nPt(0) = 0#: nPt(1) = 0#: nPt(2) = 1#
nPt(3) = -1#: nPt(4) = -1#: nPt(5) = 0#
nPt(6) = 1#: nPt(7) = -1#: nPt(8) = 0#
vPt = nPt
bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Empty)

' Left
ReDim nPt(8)
nPt(0) = 0#: nPt(1) = 0#: nPt(2) = 1#
nPt(3) = -1#: nPt(4) = -1#: nPt(5) = 0#
nPt(6) = -1#: nPt(7) = 1#: nPt(8) = 0#
vPt = nPt
bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Empty)

' Back
ReDim nPt(8)
nPt(0) = 0#: nPt(1) = 0#: nPt(2) = 1#
nPt(3) = -1#: nPt(4) = 1#: nPt(5) = 0#
nPt(6) = 1#: nPt(7) = 1#: nPt(8) = 0#
vPt = nPt
bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Empty)

' Right
ReDim nPt(8)
nPt(0) = 0#: nPt(1) = 0#: nPt(2) = 1#
nPt(3) = 1#: nPt(4) = 1#: nPt(5) = 0#
nPt(6) = 1#: nPt(7) = -1#: nPt(8) = 0#
vPt = nPt
bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Empty)

' Bottom
ReDim nPt(11)
nPt(0) = -1#: nPt(1) = -1#: nPt(2) = 0#
nPt(3) = -1#: nPt(4) = 1#: nPt(5) = 0#
nPt(6) = 1#: nPt(7) = 1#: nPt(8) = 0#
nPt(9) = 1#: nPt(10) = -1#: nPt(11) = 0#
vPt = nPt
bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Empty)

bRet = swBody.CreateBodyFromSurfaces

swModel.ViewZoomtofit2
```

End Sub

Create Imported Solid Body Example (VB.NET)

This example shows how to create an imported solid body in the shape of a pyramid.

```

'-----
' Preconditions: Verify that the specified part
' document template exists.
' Postconditions:
' 1. Opens a new part document.
' 2. Creates a pyramid-shaped, imported, solid body.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```
Partial Class SolidWorksMacro
```

```

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swPart As PartDoc
        Dim swBody As Body2
        Dim nPt() As Double
        Dim vPt As Object
        Dim bRet As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\part.prtdot", 0, 0, 0)
        swPart = swModel
        swBody = swPart.CreateNewBody
        ' Front
        ReDim nPt(8)
        nPt(0) = 0.0# : nPt(1) = 0.0# : nPt(2) = 1.0#
        nPt(3) = -1.0# : nPt(4) = -1.0# : nPt(5) = 0.0#
        nPt(6) = 1.0# : nPt(7) = -1.0# : nPt(8) = 0.0#
        vPt = nPt
        bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Nothing)
        ' Left
        ReDim nPt(8)
        nPt(0) = 0.0# : nPt(1) = 0.0# : nPt(2) = 1.0#
        nPt(3) = -1.0# : nPt(4) = -1.0# : nPt(5) = 0.0#
        nPt(6) = -1.0# : nPt(7) = 1.0# : nPt(8) = 0.0#
        vPt = nPt
        bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Nothing)
        ' Back
        ReDim nPt(8)
        nPt(0) = 0.0# : nPt(1) = 0.0# : nPt(2) = 1.0#
        nPt(3) = -1.0# : nPt(4) = 1.0# : nPt(5) = 0.0#
        nPt(6) = 1.0# : nPt(7) = 1.0# : nPt(8) = 0.0#
        vPt = nPt
        bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Nothing)
        ' Right
        ReDim nPt(8)
        nPt(0) = 0.0# : nPt(1) = 0.0# : nPt(2) = 1.0#
        nPt(3) = 1.0# : nPt(4) = 1.0# : nPt(5) = 0.0#
        nPt(6) = 1.0# : nPt(7) = -1.0# : nPt(8) = 0.0#
        vPt = nPt
        bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Nothing)
        ' Bottom
        ReDim nPt(11)
        nPt(0) = -1.0# : nPt(1) = -1.0# : nPt(2) = 0.0#
        nPt(3) = -1.0# : nPt(4) = 1.0# : nPt(5) = 0.0#
        nPt(6) = 1.0# : nPt(7) = 1.0# : nPt(8) = 0.0#
        nPt(9) = 1.0# : nPt(10) = -1.0# : nPt(11) = 0.0#
        vPt = nPt
        bRet = swBody.CreatePlanarTrimSurfaceDLL((vPt), Nothing)

        bRet = swBody.CreateBodyFromSurfaces

        swModel.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

```
End Class
```

Create Imported Surface Body from Sketch Example (C#)

This example shows how to create an imported body surface from a sketch.

```
'-----
'
' Preconditions: A circular sketch is selected.
'
' Postconditions: A cylindrical imported surface body is created.
'
'-----

using System;
namespace BodyFromSketchProfile
{
    /// <summary>
    /// Summary description for Class1
    /// </summary>
    class Application
    {
        static double ExtrudeDepth = 0.1;
        /// <summary>
        /// Main entry point for the application
        /// </summary>
        [STAThread]
        static void Main(string[] args)
        {
            bool bRetVal;
            SldWorks.ISldWorks swApp;
            SldWorks.IModelDoc2 swDoc;
            SldWorks.IPartDoc swPart;
            SldWorks.IBody2 swBody;
            SldWorks.ISketch swSketch;
            SldWorks.IFeature swFeature;

            swApp = new SldWorks.SldWorksClass();
            swDoc = swApp.IActiveDoc2;
            swPart = (SldWorks.IPartDoc) swDoc;

            SldWorks.ISelectionMgr swSelMgr;
            swSelMgr = swDoc.ISelectionManager;
            swFeature = (SldWorks.IFeature) swSelMgr.GetSelectedObject6(1,0);
            swSketch = (SldWorks.ISketch) swFeature.GetSpecificFeature2();
            if (swSketch != null)
            {
                SldWorks.IEnumSketchSegments swSegs;
                swSegs = swSketch.IEnumSketchSegments();

                int nFetched = 0;
                int segcount = 0;
                SldWorks.SketchSegment swSeg;
                System.Collections.ArrayList ProfileSegs = new System.Collections.ArrayList();
                System.Collections.ArrayList BodySurfaces = new System.Collections.ArrayList();
                SldWorks.ISurface swSurface;

                swSegs.Next(1, out swSeg, ref nFetched);
                while (nFetched != 0)
                {
                    if (swSeg.ConstructionGeometry == false)
                    {
                        swBody = swPart.ICreateNewBody2();

                        segcount++;
                        SldWorks.ISketchPoint swStartPt;
                        SldWorks.ISketchPoint swEndPt;
                        SldWorks.ICurve swCurve = null;

                        switch (swSeg.GetType())
```

```

{
    case (int)SwConst.swSketchSegments_e.swSketchARC:
    {
        SldWorks.ISketchArc swSkArc;
        swSkArc = (SldWorks.ISketchArc) swSeg;

        SldWorks.ISketchPoint swCenterPt;
        double [] normal;
        double radius;

        swCenterPt = swSkArc.IGetCenterPoint2();
        swStartPt = swSkArc.IGetStartPoint2();
        swEndPoint = swSkArc.IGetEndPoint2();
        normal = (double[])swSkArc.GetNormalVector();
        radius = swSkArc.GetRadius();

        double []center = new double[3];
        double []startpt = new double[3];
        double []endpt = new double[3];

        center[0] = swCenterPt.X;
        center[1] = swCenterPt.Y;
        center[2] = swCenterPt.Z;
        startpt[0] = swStartPt.X;
        startpt[1] = swStartPt.Y;
        startpt[2] = swStartPt.Z;
        endpt[0] = swEndPoint.X;
        endpt[1] = swEndPoint.Y;
        endpt[2] = swEndPoint.Z;

        swCurve = swBody.IAddProfileArc(center, normal, radius, startpt, endpt);
        break;
    }
    case (int)SwConst.swSketchSegments_e.swSketchLINE:
    {
        SldWorks.ISketchLine swSkLine;
        swSkLine = (SldWorks.ISketchLine) swSeg;

        swStartPt = swSkLine.IGetStartPoint2();
        swEndPoint = swSkLine.IGetEndPoint2();

        double []root = new double[3];
        double []dir = new double[3];

        root[0] = swStartPt.X;
        root[1] = swStartPt.Y;
        root[2] = swStartPt.Z;

        dir[0] = swEndPoint.X - root[0];
        dir[1] = swEndPoint.Y - root[1];
        dir[2] = swEndPoint.Z - root[2];

        swCurve = swBody.IAddProfileLine(root, dir);

        break;
    }
    default:
        System.Diagnostics.Debug.Assert(false, "Unhandled sketch segment type");
        break;
}

// Have the curve and the start and end points
// Create the side of the extrusion

ProfileSegment curSeg = new ProfileSegment(swSeg, swCurve);
double []surfeval;
double []extruAxis = new double[3];

extruAxis[0] = 0.0;
extruAxis[1] = 0.0;
extruAxis[2] = 1.0;

ProfileSegs.Add(curSeg);

```

```

        swSurface = swBody.ICreateExtrusionSurface((SldWorks.Curve)swCurve, extruAxis);

        //Create a trimming loop for the surface
        int nCurves = 4;
        int []Order = {2, 2, 2, 2};
        int []Dim = {2, 2, 2, 2};
        int []Periodic = {0, 0, 0, 0};
        int []nKnots = {4, 4, 4, 4};
        int []nCtrlPts = {2, 2, 2, 2};
        double []Knots = {0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1};
        double []CtrlPts = new double[16];

        double x = curSeg.iStartPt.X;
        double y = curSeg.iStartPt.Y;
        double z = curSeg.iStartPt.Z;
        surfeval = (double[])swSurface.GetClosestPointOn(x, y, z);
        CtrlPts[0] = surfeval[3];
        CtrlPts[1] = surfeval[4];
        CtrlPts[14] = surfeval[3];
        CtrlPts[15] = surfeval[4];

        x = curSeg.iEndPt.X;
        y = curSeg.iEndPt.Y;
        z = curSeg.iEndPt.Z;
        surfeval = (double[])swSurface.GetClosestPointOn(x, y, z);
        CtrlPts[2] = surfeval[3];
        CtrlPts[3] = surfeval[4];
        CtrlPts[4] = surfeval[3];
        CtrlPts[5] = surfeval[4];

        x = curSeg.iEndPt.X;
        y = curSeg.iEndPt.Y;
        z = curSeg.iEndPt.Z;
        surfeval = (double[])swSurface.GetClosestPointOn(x, y, z + ExtrudeDepth);
        CtrlPts[6] = surfeval[3];
        CtrlPts[7] = surfeval[4];
        CtrlPts[8] = surfeval[3];
        CtrlPts[9] = surfeval[4];

        x = curSeg.iStartPt.X;
        y = curSeg.iStartPt.Y;
        z = curSeg.iStartPt.Z;
        surfeval = (double[])swSurface.GetClosestPointOn(x, y, z + ExtrudeDepth);
        CtrlPts[10] = surfeval[3];
        CtrlPts[11] = surfeval[4];
        CtrlPts[12] = surfeval[3];
        CtrlPts[13] = surfeval[4];

        swSurface.AddTrimmingLoop(nCurves, Order, Dim, Periodic, nKnots, nCtrlPts, Knots, CtrlPts);

        bRetval = swBody.CreateTrimmedSurface();
        bRetval = swBody.CreateBodyFromSurfaces();
    }
    swSegs.Next(1, out swSeg, ref nFetched);
}

}
} // Main
} // Application
class ProfileSegment
{
    public SldWorks.ISketchSegment iSkSeg;
    public SldWorks.ICurve iCurve;
    public SldWorks.ISketchPoint iStartPt;
    public SldWorks.ISketchPoint iEndPt;
    public ProfileSegment(SldWorks.ISketchSegment swSkSeg, SldWorks.ICurve swCurve)
    {
        iSkSeg = swSkSeg;
        iCurve = swCurve;

        switch (iSkSeg.GetType())
        {

```

```

        case (int)SwConst.swSketchSegments_e.swSketchARC:
        {
            SldWorks.ISketchArc swSkArc;
            swSkArc = (SldWorks.ISketchArc) iSkSeg;

            iStartPt = swSkArc.IGetStartPoint2();
            iEndPt = swSkArc.IGetEndPoint2();
            break;
        }

        case (int)SwConst.swSketchSegments_e.swSketchLINE:
        {
            SldWorks.ISketchLine swSkLine;
            swSkLine = (SldWorks.ISketchLine) iSkSeg;

            iStartPt = swSkLine.IGetStartPoint2();
            iEndPt = swSkLine.IGetEndPoint2();

            break;
        }
        default:
            System.Diagnostics.Debug.Assert(false, "Unhandled sketch segment type");
            break;
    }
}

class Vector
{
    private
        double []coords = new double[3];

    public Vector(double x, double y, double z)
    {
        coords[0] = x; coords[1] = y; coords[2] = z;
    }
}

}
// BodyFromSketchProfile

```

Create Infinite Plane Example (VBA)

This example shows how to create an infinite plane.

```

'-----
' Preconditions:
' 1. Open a model document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates an infinite plane.
' 2. Examine the Immediate window.
'-----
Option Explicit

' Define two types
Type DoubleRec
    dValue As Double
End Type

Type Long2Rec
    iLower As Long
    iUpper As Long
End Type

' Extract two integer values from a single double value
' by assigning a DoubleRec to the double value and
' copying the value to a Long2Rec and
' extracting the integer values
Function ExtractFields(ByVal dValue As Double, iLower As Integer, iUpper As Integer)
    Dim dr As DoubleRec
    Dim i2r As Long2Rec
    ' Set the double value
    dr.dValue = dValue
    ' Copy the values
    LSet i2r = dr
    ' Extract the values
    iLower = i2r.iLower
    iUpper = i2r.iUpper
End Function

Sub ProcessSurface(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swSurf As SldWorks.Surface)
    Dim vSurfParam As Variant
    Dim uBboundType(2) As Integer
    Dim vBboundType(2) As Integer
    Dim Num_uProp As Integer
    Dim Num_vProp As Integer
    Dim uProp(4) As Integer
    Dim vProp(4) As Integer
    Dim i As Integer

    vSurfParam = swSurf.Parameterization
    ExtractFields vSurfParam(4), uBboundType(1), uBboundType(2)
    ExtractFields vSurfParam(5), vBboundType(1), vBboundType(2)
    ExtractFields vSurfParam(10), Num_uProp, Num_vProp
    ExtractFields vSurfParam(6), uProp(1), uProp(2)
    ExtractFields vSurfParam(7), uProp(3), uProp(4)
    ExtractFields vSurfParam(8), vProp(1), vProp(2)
    ExtractFields vSurfParam(9), vProp(3), vProp(4)

    Debug.Print " uRange      = [" & vSurfParam(0) & " , " & vSurfParam(1) & "]"
    Debug.Print " uBoundType   = [" & uBboundType(1) & " , " & uBboundType(2) & "]"
    Debug.Print " Num_uProp    = " & Num_uProp

    For i = 1 To Num_uProp
        Debug.Print "    uProp[" & i & "] = " & uProp(i)
    Next i

    Debug.Print ""

    Debug.Print " vRange      = [" & vSurfParam(2) & " , " & vSurfParam(3) & "]"
    Debug.Print " vBoundType  = [" & vBboundType(1) & " , " & vBboundType(2) & "]"
    Debug.Print " Num_vProp   = " & Num_vProp

    For i = 1 To Num_vProp
        Debug.Print "    vProp[" & i & "] = " & vProp(i)
    Next i
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModeller As SldWorks.Modeler
    Dim swSurf As SldWorks.Surface
    Dim BasePt(2) As Double
    Dim Normal(2) As Double
    Dim Xvect(2) As Double
    Dim VBasePt As Variant
    Dim vNormal As Variant
    Dim vXvect As Variant
    Dim vPlane As Variant
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModeller = swApp.GetModeler

```

```

BasePt(0) = 0#:      BasePt(1) = 0#:      BasePt(2) = 0#
vBasePt = BasePt

Normal(0) = 0#:      Normal(1) = 0#:      Normal(2) = 1#
vNormal = Normal

Xvect(0) = 1#:      Xvect(1) = 0#:      Xvect(2) = 0#
vXvect = Xvect

Set swSurf = swModeller.CreatePlanarSurface2((vBasePt), (vNormal), (vXvect))

Debug.Assert Not swSurf Is Nothing
Debug.Assert swSurf.IsPlane

vPlane = swSurf.PlaneParams
Debug.Print "SW Version = " & swApp.RevisionNumber
Debug.Print "  normal      = (" & vPlane(0) & ", " & vPlane(1) & ", " & vPlane(2) & ")"
Debug.Print "  root        = (" & vPlane(3) * 1000# & ", " & vPlane(4) * 1000# & ", " & vPlane(5) * 1000# & ") mm"

Debug.Print ""

ProcessSurface swApp, Nothing, swSurf

End Sub

```

Create Intersect Feature Example (C#)

This example shows how to create an intersect feature that includes only the internal (hollow) regions between the intersecting regions.

```
//-----
// Preconditions: Verify that the specified part exists.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Selects Shell1 and Plane6.
// 3. Creates Intersect1 feature.
// 4. Examine the FeatureManager design tree.
//
// NOTE: Because the part is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureMgr = default(FeatureManager);
            Feature swFeature = default(Feature);
            object[] bodyArray = null;
            bool[] bodyExcludeArray = null;
            object excludeArray = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int nbrBodies = 0;
            int i = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2017\\whatsnew\\parts\\pot.SLDPRT";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;

            //Select features for intersect feature
            status = swModelDocExt.SelectByID2("Shell1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Plane6", "PLANE", 0, 0, 0, true, 0, null, 0);

            // Create intersect feature
            // * Do not cap planar surface openings of intersect feature
            // * Create internal regions
            bodyArray = (object[])swFeatureMgr.PreIntersect2(false, 1);
            swModel.ClearSelection2(true);

            nbrBodies = bodyArray.GetUpperBound(0);
            bodyExcludeArray = new bool[nbrBodies + 1];
            for (i = 0; i <= nbrBodies; i++)
            {
                bodyExcludeArray[i] = false;
            }
            excludeArray = bodyExcludeArray;
            swFeature = swFeatureMgr.PostIntersect(excludeArray, true, false);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Intersect Feature Example (VBA)

This example shows how to create an intersect feature that includes only the internal (hollow) regions between the intersecting regions.

```
'-----
' Preconditions: Verify that the specified part exists.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Selects Shell1 and Plane6.
' 3. Creates Intersect1 feature.
' 4. Examine the FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim bodyArray As Variant
Dim bodyExcludeArray() As Boolean
Dim excludeArray As Variant
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim nbrBodies As Long
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\whatsnew\parts\pot.SLDPRPT"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureMgr = swModel.FeatureManager

    'Select features for intersect feature
    status = swModelDocExt.SelectByID2("Shell1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Plane6", "PLANE", 0, 0, 0, True, 0, Nothing, 0)

    ' Create intersect feature
    ' * Do not cap planar surface openings of intersect feature
    ' * Create internal regions
    bodyArray = swFeatureMgr.PreIntersect2(False, 1)
    swModel.ClearSelection2 True

    nbrBodies = UBound(bodyArray)
    ReDim bodyExcludeArray(nbrBodies)
    For i = 0 To nbrBodies
        bodyExcludeArray(i) = False
    Next
    excludeArray = bodyExcludeArray
    Set swFeature = swFeatureMgr.PostIntersect(excludeArray, True, False)

End Sub
```

Create Intersect Feature Example (VB.NET)

This example shows how to create an intersect feature that includes only the internal (hollow) regions between the intersecting regions.

```

-----
Preconditions: Verify that the specified part exists.
-----
Postconditions:
1. Opens the specified part.
2. Selects Shell1 and Plane6.
3. Creates Intersect1 feature.
4. Examine the FeatureManager design tree.
-----
NOTE: Because the part is used elsewhere, do not save
changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureMgr As FeatureManager
        Dim swFeature As Feature
        Dim bodyArray As Object
        Dim bodyExcludeArray() As Boolean
        Dim excludeArray As Object
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim nbrBodies As Integer
        Dim i As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\whatsnew\parts\pot.SLDPRPT"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swFeatureMgr = swModel.FeatureManager

        'Select features for intersect feature
        status = swModelDocExt.SelectByID2("Shell1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Plane6", "PLANE", 0, 0, 0, True, 0, Nothing, 0)

        ' Create intersect feature
        ' * Do not cap planar surface openings of intersect feature
        ' * Create internal regions
        bodyArray = swFeatureMgr.PreIntersect2(False, 1)
        swModel.ClearSelection2(True)

        nbrBodies = UBound(bodyArray)
        ReDim bodyExcludeArray(nbrBodies)
        For i = 0 To nbrBodies
            bodyExcludeArray(i) = False
        Next
        excludeArray = bodyExcludeArray
        swFeature = swFeatureMgr.PostIntersect(excludeArray, True, False)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create Layer for Selected View Example (C#)

This example shows how to create a layer for the part in the selected drawing view.

```
//-----
// Preconditions:
// 1. Open a drawing of a part.
// 2. Add a reference to Microsoft.VisualBasic.
// 3. Select a drawing view in the FeatureManager design tree.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Creates a layer for the part in the selected drawing view.
// 2. Click the Layer Properties tool on the Line Format toolbar to verify
//    that the newly created layer is selected in the Layers dialog box.
// 3. Examine the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace CreateLayer2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        private void ChangeComponentLayer(DrawingDoc swDraw, string sLayerName)
        {
            bool bRet = false;

            // Form a valid layer name
            sLayerName = Strings.Replace(sLayerName, "/", "_", 1, -1, 0);
            sLayerName = Strings.Replace(sLayerName, "@", "_", 1, -1, 0);

            bRet = swDraw.CreateLayer2(sLayerName, "Layer for part in " + sLayerName, 0, (int)swLineStyles_e.swLineCONTINUOUS,
            (int)swLineWeights_e.swLW_NORMAL, true, true);

            // Change in all views
            swDraw.ChangeComponentLayer(sLayerName, true);
        }

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            View swView = default(View);
            ModelDoc2 swDrawModel = default(ModelDoc2);
            PartDoc swDrawPart = default(PartDoc);
            int nErrors = 0;
            int nWarnings = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            swDrawModel = swApp.OpenDoc6(swView.GetReferencedModelName(), (int)swDocumentTypes_e.swDocPART,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref nErrors, ref nWarnings);
            swDrawPart = (PartDoc)swDrawModel;

            Debug.Print("File          = " + swModel.GetPathName());
            Debug.Print("  View          = " + swView.Name);
            Debug.Print("  View Model    = " + swView.GetReferencedModelName());

            ChangeComponentLayer(swDraw, swView.Name);
        }

        public SldWorks swApp;
    }
}
```

Create Layer for Selected View Example (VBA)

This example shows how to create a layer for the part in the selected drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing of a part.
' 2. Select a drawing view in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates a layer for the part in the selected drawing view.
' 2. Click the Layer Properties tool on the Line Format toolbar to verify
'    that the newly created layer is selected in the Layers dialog box.
' 3. Examine the Immediate window.
'-----
```

Option Explicit

```
Private Sub ChangeComponentLayer _
( _
    swApp As SldWorks.SldWorks, _
    swDraw As SldWorks.DrawingDoc, _
    sLayerName As String _
)
    Dim bRet As Boolean

    ' Form a valid layer name
    sLayerName = Replace(sLayerName, "/", "_")
    sLayerName = Replace(sLayerName, "@", "-")

    bRet = swDraw.CreateLayer2( _
        sLayerName, _
        "Layer for part in " & sLayerName, _
        0, swLineCONTINUOUS, swLW_NORMAL, True, True)

    ' Change in all drawing views
    swDraw.ChangeComponentLayer sLayerName, True
End Sub

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swView As SldWorks.View
    Dim swDrawModel As SldWorks.ModelDoc2
    Dim swDrawPart As SldWorks.PartDoc
    Dim vBody As Variant
    Dim swBody As SldWorks.Body2
    Dim swFace As SldWorks.Face2
    Dim swEnt As SldWorks.Entity
    Dim nErrors As Long
    Dim nWarnings As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    Set swDrawModel = swApp.OpenDoc6(swView.GetReferencedModelName, swDocPART, swOpenDocOptions_Silent, "", nErrors, nWarnings)
    Set swDrawPart = swDrawModel

    Debug.Print "File           = " & swModel.GetPathName
    Debug.Print "  View          = " & swView.Name
    Debug.Print "    View Model = " & swView.GetReferencedModelName

    vBody = swDrawPart.GetBodies2(swSolidBody, True)

    Set swBody = vBody(0)
    Set swFace = swBody.GetFirstFace
    Set swEnt = swFace

    bRet = swView.SelectEntity(swEnt, False)

    ChangeComponentLayer swApp, swDraw, swView.Name
End Sub
```

Create Layer for Selected View Example (VB.NET)

This example shows how to create a layer for the part in the selected drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing of a part.
' 2. Select a drawing view in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Creates a layer for the part in the selected drawing view.
' 2. Click the Layer Properties tool on the Line Format toolbar to verify
'    that the newly created layer is selected in the Layers dialog box.
' 3. Examine the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Private Sub ChangeComponentLayer _
    ( _
    ByVal swDraw As DrawingDoc, _
    ByVal sLayerName As String _
    )
        Dim bRet As Boolean

        ' Form a valid layer name
        sLayerName = Replace(sLayerName, "/", "_")
        sLayerName = Replace(sLayerName, "@", "-")

        bRet = swDraw.CreateLayer2( _
            sLayerName, _
            "Layer for part in " & sLayerName, _
            0, swLineStyle_e.swLineCONTINUOUS, swLineWeights_e.swLW_NORMAL, True, True)

        ' Change in all drawing views
        swDraw.ChangeComponentLayer(sLayerName, True)

    End Sub

    Sub main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swSelMgr As SelectionMgr
        Dim swView As View
        Dim swDrawModel As ModelDoc2
        Dim swDrawPart As PartDoc
        Dim vBody As Object
        Dim swBody As Body2
        Dim swFace As Face2
        Dim swEnt As Entity
        Dim nErrors As Long
        Dim nWarnings As Long
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swDraw = swModel
        swSelMgr = swModel.SelectionManager
        swView = swSelMgr.GetSelectedObject6(1, -1)
        swDrawModel = swApp.OpenDoc6(swView.GetReferencedModelName, swDocumentTypes_e.swDocPART,
        swOpenDocOptions_e.swOpenDocOptions_Silent, "", nErrors, nWarnings)
        swDrawPart = swDrawModel

        Debug.Print("File      = " & swModel.GetPathName)
        Debug.Print("  View      = " & swView.Name)
        Debug.Print("  View Model = " & swView.GetReferencedModelName)

        vBody = swDrawPart.GetBodies2(swBodyType_e.swSolidBody, True)

        swBody = vBody(0)
        swFace = swBody.GetFirstFace
        swEnt = swFace

        bRet = swView.SelectEntity(swEnt, False)

        ChangeComponentLayer(swDraw, swView.Name)

    End Sub

    Public swApp As SldWorks
```

End Class

Create Library Feature Data Object and Library Feature With References Example (C#)

This example shows how to create a library feature with references in order to position the library feature on a model.

```
//-----
// Preconditions:
// 1. Verify that the specified part template and library feature
// exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a new part containing a boss extrude.
// 2. Creates a library feature data object.
// a. Initializes the newly created library feature using
// the specified library feature.
// b. Gets the type of references required for the library
// feature.
// c. Sets the name of the active library feature configuration.
// d. Selects the face where to create the library feature.
// e. Creates the library feature.
// f. Accesses the library feature and selects the edges where to
// position the it.
// g. Sets the references for positioning the library feature.
// h. Updates the definition of the library feature.
// i. Unsuppresses the library feature.
// 3. Examine the Immediate window, FeatureManager design tree, and
// graphics area.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            Feature swFeature = default(Feature);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            FeatureManager swFeatureManager = default(FeatureManager);
            LibraryFeatureData swLibFeat = default(LibraryFeatureData);
            bool status = false;
            object[] sketchLines = null;
            object Refs = null;
            object RefTypes = null;
            int RefCount = 0;
            int k = 0;
            int i = 0;
            DispatcherWrapper[] LibRefs = null;

            // Create part
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDet
            ailingNoOptionSpecified, false);
            status = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.s
            wDetailingNoOptionSpecified, true);
            swSketchManager = (SketchManager)swModel.SketchManager;
            sketchLines = (object[])swSketchManager.CreateCornerRectangle(0, 0, 0, 1, 0.5, 0);
            swModel.ShowNamedView2("Trimetric", 8);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.01, 0.01, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
            0, 0, false);
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            swSelectionManager.EnableContourSelection = false;

            // Create library feature
            swLibFeat = (LibraryFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmLibraryFeature);

            // Initialize newly created library feature using the specified library part
            status = swLibFeat.Initialize("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\design library\\features\\metric\\slots\\straight slot.sldlfp");

            // Get the type of references required for the library feature
            RefCount = swLibFeat.GetReferencesCount();
            Refs = (object[])swLibFeat.GetReferences2((int)swLibFeatureData_e.swLibFeatureData_FeatureRespect, out RefTypes);

            if ((RefTypes != null))
            {
                Debug.Print("Types of references required (edge = 1): ");
                int[] RefType = (int[])RefTypes;
                for (k = RefType.GetLowerBound(0); k <= RefType.GetUpperBound(0); k++)
                {
                    Debug.Print("    " + RefType[k].ToString());
                }
            }

            // Set the name of the active library feature configuration
            swLibFeat.ConfigurationName = "Default";

            // Select the face where to create the library feature
            status = swModelDocExt.SelectByID2("", "FACE", 0.522458766456054, 0.288038964184011, 0.00999999999987722, false, 0, null, 0);

            // Create the library feature
            swFeature = (Feature)swFeatureManager.CreateFeature(swLibFeat);
        }
    }
}
```

```

// Access the library feature to position it on the part
swLibFeat = null;
swLibFeat = (LibraryFeatureData)swFeature.GetDefinition();
status = swLibFeat.AccessSelections(swModel, null);

// Select the edges where to position the library feature
status = swModelDocExt.SelectByID("", "EDGE", 0.960865149149924, 0.497807163546383, 0.0131011390528215, true, 0, null, 0);
status = swModelDocExt.SelectByID("", "EDGE", 0.99866860703213, 0.481385806014544, 0.0113313929676906, true, 0, null, 0);

int selCount = 0;
selCount = swSelectionManager.GetSelectedObjectCount2(-1);

object[] selectedObjects = new object[selCount];

for (i = 0; i < selCount; i++)
{
    object selectedObject = null;
    selectedObject = (object)swSelectionManager.GetSelectedObject6(i + 1, -1);
    selectedObjects[i] = selectedObject;
}

// Convert the .NET array to IDispatch
LibRefs = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray((selectedObjects));

// Set the references for positioning the library feature on the part
swLibFeat.SetReferences(LibRefs);

// Update the definition of the library feature
status = swFeature.ModifyDefinition(swLibFeat, swModel, null);

// Unsuppress the library feature
status = swModelDocExt.SelectByID("straight slot<1>", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swModel.EditUnsuppress2();

swModel.ClearSelection2(true);
}

public DispatchWrapper[] ObjectArrayToDispatchWrapperArray(object[] Objects)
{
    int ArraySize = 0;
    ArraySize = Objects.GetUpperBound(0);
    DispatchWrapper[] d = new DispatchWrapper[ArraySize + 1];
    int ArrayIndex = 0;
    for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
    {
        d[ArrayIndex] = new DispatchWrapper(Objects[ArrayIndex]);
    }
    return d;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
public DispatchWrapper[] LibRefs;
}
}

```

Create Library Feature Data Object and Library Feature With References Example (VBA)

This example shows how to create a library feature with references in order to position the library feature on a model.

```

'-----
' Preconditions:
' 1. Verify that the specified part template and library feature
'    exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part containing a boss extrude.
' 2. Creates a library feature data object.
'   a. Initializes the newly created library feature using
'      the specified library feature.
'   b. Gets the type of references required for the library
'      feature.
'   c. Sets the name of the active library feature configuration.
'   d. Selects the face where to create the library feature.
'   e. Creates the library feature.
'   f. Accesses the library feature and selects the edges where to
'      position the it.
'   g. Sets the references for positioning the library feature.
'   h. Updates the definition of the library feature.
'   i. Unsuppresses the library feature.
' 3. Examine the Immediate window, FeatureManager design tree, and
'    graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swFeature As SldWorks.Feature
Dim swFeatureManager As SldWorks.FeatureManager
Dim swSelectionManager As SldWorks.SelectionMgr
Dim swLibFeat As SldWorks.LibraryFeatureData
Dim sketchLines As Variant
Dim status As Boolean
Dim obj() As Object
Dim vRefs As Variant
Dim vRefTypes As Variant
Dim refType As Variant
Dim nRefCount As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Create part
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Set swSketchManager = swModel.SketchManager
    sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 1, 0.5, 0)

    swModel.ShowNamedView2 "Trimetric", 8
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.01, 0.01, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)
    Set swSelectionManager = swModel.SelectionManager
    swSelectionManager.EnableContourSelection = False

    ' Create library feature data object
    Set swLibFeat = swFeatureManager.CreateDefinition(swFmLibraryFeature)

    ' Initialize newly created library feature using the specified library part
    status = swLibFeat.Initialize("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\design library\features\metric\slots\straight slot.sldlfp")

    ' Get the type of references required for the library feature
    nRefCount = swLibFeat.GetReferencesCount
    vRefs = swLibFeat.GetReferences2(swLibFeatureData_FeatureRespect, vRefTypes)
    If Not IsEmpty(vRefTypes) Then
        Debug.Print "Types of references required (edge = 1): "
        For Each refType In vRefTypes
            Debug.Print " " & CStr(refType)
        Next
    End If

    ' Set the name of the active library feature configuration
    swLibFeat.ConfigurationName = "Default"

    ' Select the face where to create the library feature
    status = swModelDocExt.SelectByID2("", "FACE", 0.522458766456054, 0.288038964184011, 9.9999999987722E-03, False, 0, Nothing, 0)

```

```

' Create the library feature
Set swFeature = swFeatureManager.CreateFeature(swLibFeat)

' Access the library feature to position it on the part
Set swLibFeat = Nothing
Set swLibFeat = swFeature.GetDefinition
status = swLibFeat.AccessSelections(swModel, Nothing)

' Select the edges where to position the library feature
status = swModelDocExt.SelectByID("", "EDGE", 0.960865149149924, 0.497807163546383, 1.31011390528215E-02, True, 0, Nothing, 0)
status = swModelDocExt.SelectByID("", "EDGE", 0.99866860703213, 0.481385806014544, 1.13313929676906E-02, True, 0, Nothing, 0)

Dim selCount As Long
selCount = swSelectionManager.GetSelectedObjectCount2(-1)
selCount = selCount - 1
ReDim obj(selCount) As Object
Dim i As Long
For i = 0 To selCount
    Set obj(i) = swSelectionManager.GetSelectedObject6(i + 1, -1)
Next

' Set the references for positioning the library feature on the part
Dim vLibRefs As Variant
vLibRefs = obj
swLibFeat.SetReferences (vLibRefs)

' Update the definition of the library feature
status = swFeature.ModifyDefinition(swLibFeat, swModel, Nothing)

' Unsuppress the library feature
status = swModelDocExt.SelectByID("straight slot<1>", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swModel.EditUnsuppress2

swModel.ClearSelection2 True

End Sub

```

Create Library Feature Data Object and Library Feature With References Example (VB.NET)

This example shows how to create a library feature with references in order to position the library feature on a model.

```

-----
Preconditions:
1. Verify that the specified part template and library feature exist.
2. Open the Immediate window.

Postconditions:
1. Creates a new part containing a boss extrude.
2. Creates a library feature data object.
   a. Initializes the newly created library feature using the specified library feature.
   b. Gets the type of references required for the library feature.
   c. Sets the name of the active library feature configuration.
   d. Selects the face where to create the library feature.
   e. Creates the library feature.
   f. Accesses the library feature and selects the edges where to position the it.
   g. Sets the references for positioning the library feature.
   h. Updates the definition of the library feature.
   i. Unsuppresses the library feature.
3. Examine the Immediate window, FeatureManager design tree, and graphics area.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics
Imports System.Runtime.InteropServices

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swFeature As Feature
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSelectionManager As SelectionMgr
        Dim swFeatureManager As FeatureManager
        Dim swLibFeat As LibraryFeatureData
        Dim status As Boolean
        Dim sketchLines() As Object
        Dim selectedObjects() As Object
        Dim Refs As Object = Nothing
        Dim RefTypes As Object = Nothing
        Dim RefType As Object
        Dim RefCount As Integer
        Dim LibRefs() As DispatcherWrapper

        ' Create part
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
        status = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
        swSketchManager = swModel.SketchManager
        sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 1, 0.5, 0)
        swModel.ShowNamedView2("Trimetric", 8)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.01, 0.01, False, False, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, True, True, 0, False)
        swSelectionManager = swModel.SelectionManager
        swSelectionManager.EnableContourSelection = False

        ' Create library feature
        swLibFeat = swFeatureManager.CreateDefinition(swFeatureNameID_e.swFmLibraryFeature)

        ' Initialize newly created library feature using the specified library part
        status = swLibFeat.Initialize("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\design library\features\metric\slots\straight slot.sldlfp")

        ' Get the type of references required for the library feature
        RefCount = swLibFeat.GetReferencesCount
        Refs = swLibFeat.GetReferences2(swLibFeatureData_e.swLibFeatureData_FeatureRespect, RefTypes)

        If Not IsNothing(RefTypes) Then
            Debug.Print("Types of references required (edge = 1): ")
            For Each RefType In RefTypes
                Debug.Print(vbTab + CStr(RefType))
            Next
        End If

        ' Set the name of the active library feature configuration
        swLibFeat.ConfigurationName = "Default"

        ' Select the face where to create the library feature
        status = swModelDocExt.SelectByID2("", "FACE", 0.522458766456054, 0.288038964184011, 0.00999999999997722, False, 0, Nothing, 0)

        ' Create the library feature
        swFeature = swFeatureManager.CreateFeature(swLibFeat)

        ' Access the library feature to position it on the part
        swLibFeat = Nothing
        swLibFeat = swFeature.GetDefinition
        status = swLibFeat.AccessSelections(swModel, Nothing)

        ' Select the edges where to position the library feature
    
```

```
status = swModelDocExt.SelectByID("", "EDGE", 0.960865149149924, 0.497807163546383, 0.0131011390528215, True, 0, Nothing, 0)
status = swModelDocExt.SelectByID("", "EDGE", 0.99866860703213, 0.481385806014544, 0.0113313929676906, True, 0, Nothing, 0)
```

```
Dim selCount As Integer
selCount = swSelectionManager.GetSelectedObjectCount2(-1)

ReDim selectedObjects(selCount)

Dim i As Integer
For i = 0 To (selCount - 1)
    selectedObjects(i) = swSelectionManager.GetSelectedObject6(i + 1, -1)
Next i
```

```
ReDim Preserve selectedObjects(selCount - 1)

' Convert the .NET array to IDispatch
LibRefs = ObjectArrayToDispatchWrapperArray(selectedObjects)

' Set the references for positioning the library feature on the part
swLibFeat.SetReferences(LibRefs)

' Update the definition of the library feature
status = swFeature.ModifyDefinition(swLibFeat, swModel, Nothing)

' Unsuppress the library feature
status = swModelDocExt.SelectByID("straight slot<1>", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swModel.EditUnsuppress2()

swModel.ClearSelection2(True)
```

End Sub

```
Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As Dispatcher()
    Dim ArraySize As Integer
    ArraySize = Objects.GetUpperBound(0)
    Dim d(ArraySize) As Dispatcher
    Dim ArrayIndex As Integer
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New Dispatcher(Objects(ArrayIndex))
    Next
    Return d
End Function
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Create and Edit Limit Angle Mate Example (C#)

This example shows how to create and edit a limit angle advanced mate.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo1.sldasm
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a limit angle mate.
// 2. Press F5 to continue.
// 3. Modifies the mate alignment of the limit angle mate.
// 4. Inspect the Immediate window and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace LimitAngleMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private AssemblyDoc swAssy;
        private AngleMateFeatureData swAngleMateData;
        private MateFeatureData swMateData;
        private SelectionMgr swSelMgr;
        private bool boolstatus;
        private Feature feat;
        private Face2[] facesDist = new Face2[2];
        private object vFacesDist;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            swMateData = (MateFeatureData)swAssy.CreateMateData(6); //Angle mate
            Debug.Print("Type of mate as defined in swMateType_e: " + swMateData.TypeName);

            swAngleMateData = (AngleMateFeatureData)swMateData;
            swAngleMateData.MateAlignment = (int)swMateAlign_e.swMateAlignALIGNED;
            swAngleMateData.FlipDimension = false;
            swAngleMateData.Angle = 0.523599;
            swAngleMateData.MaximumAngle = 0.523599;
            swAngleMateData.MinimumAngle = 0.506145;

            boolstatus = swModel.Extension.SelectByRay(0.00868857956595548, 0.0414144214960288, 0.0633435410960033, -0.5201487747284);
            boolstatus = swModel.Extension.SelectByRay(0.0313565896258297, 0.0296508617577445, 0.0442099188854286, 0.340524666870961);

            facesDist[0] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            facesDist[1] = (Face2)swSelMgr.GetSelectedObject6(2, -1);

            vFacesDist = facesDist;
            swAngleMateData.EntitiesToMate = vFacesDist;

            feat = (Feature)swAssy.CreateMate(swAngleMateData);

            swModel.GraphicsRedraw2();

            System.Diagnostics.Debugger.Break();

            feat = swModel.Extension.GetLastFeatureAdded();
            Debug.Print("Feature type created is " + feat.GetTypeName2());

            swMateData = (MateFeatureData)feat.GetDefinition();
            swAngleMateData = (AngleMateFeatureData)swMateData;
```

```

//Change alignment of limit angle mate
if (swMateData.TypeName == (int)swMateType_e.swMateANGLE)
{

    Debug.Print("Angle mate alignment is " + swAngleMateData.MateAlignment);

    if (swAngleMateData.MateAlignment == (int)swMateAlign_e.swMateAlignALIGNED)
    {
        swAngleMateData.MateAlignment = (int)swMateAlign_e.swMateAlignANTI_ALIGNED;
    }
    else
    {
        swAngleMateData.MateAlignment = (int)swMateAlign_e.swMateAlignALIGNED;
    }

    Debug.Print("Angle mate alignment changed to " + swAngleMateData.MateAlignment);
    Debug.Print("Angle: " + swAngleMateData.Angle);
    Debug.Print("Minimum angle: " + swAngleMateData.MinimumAngle);
    Debug.Print("Maximum angle: " + swAngleMateData.MaximumAngle);
    Debug.Print("Move entities to opposite sides of the dimension? " + swAngleMateData.FlipDimension);

    boolstatus = feat.ModifyDefinition(swAngleMateData, swAssy, null);

}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

}

```

Create and Edit Limit Angle Mate Example (VBA)

This example shows how to create and edit a limit angle mate.

```

'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo1.sldasm
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a limit angle mate.
' 2. Press F5 to continue.
' 3. Modifies the mate alignment of the limit angle mate.
' 4. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swAngleMateData As SldWorks.AngleMateFeatureData
Dim swMateData As SldWorks.MateFeatureData
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim feat As SldWorks.Feature
Dim facesDist(1) As SldWorks.Face2
Dim vFacesDist As Variant

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager

    Set swMateData = swAssy.CreateMateData(6) 'Angle mate

    Debug.Print "Type of mate as defined in swMateType_e: " & swMateData.TypeName

    Set swAngleMateData = swMateData
    swAngleMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED
    swAngleMateData.FlipDimension = False
    swAngleMateData.Angle = 0.523599
    swAngleMateData.MaximumAngle = 0.523599
    swAngleMateData.MinimumAngle = 0.506145

    boolstatus = swModel.Extension.SelectByRay(8.68857956595548E-03, 4.14144214960288E-02, 6.33435410960033E-02,
    -0.520148774728431, -0.59141018013918, -0.616181183562315, 4.68381592786756E-04, 2, True, 1, 0)
    boolstatus = swModel.Extension.SelectByRay(3.13565896258297E-02, 2.96508617577445E-02, 4.42099188854286E-02, 0.340524666870961,
    -0.380278973953885, -0.859901653226112, 4.31713609895031E-04, 2, True, 1, 0)

    Set facesDist(0) = swSelMgr.GetSelectedObject6(1, -1)
    Set facesDist(1) = swSelMgr.GetSelectedObject6(2, -1)

    vFacesDist = facesDist
    swAngleMateData.EntitiesToMate = vFacesDist

    Set feat = swAssy.CreateMate(swAngleMateData)

    swModel.GraphicsRedraw2

    Stop

    Set feat = swModel.Extension.GetLastFeatureAdded
    Debug.Print "Feature type created is " & feat.GetTypeName2

    Set swMateData = feat.GetDefinition
    Set swAngleMateData = swMateData

    'Change alignment of limit angle mate
    If (swMateData.TypeName = SwConst.swMateType_e.swMateANGLE) Then

        Debug.Print "Angle mate alignment is " & swAngleMateData.MateAlignment

        If (swAngleMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED) Then
            swAngleMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignANTI_ALIGNED
        Else
            swAngleMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED
        End If
    End If

```

```
Debug.Print "Angle mate alignment changed to " & swAngleMateData.MateAlignment  
Debug.Print "Angle: " & swAngleMateData.Angle  
Debug.Print "Minimum angle: " & swAngleMateData.MinimumAngle  
Debug.Print "Maximum angle: " & swAngleMateData.MaximumAngle  
Debug.Print "Move entities to opposite sides of the dimension? " & swAngleMateData.FlipDimension
```

```
boolstatus = feat.ModifyDefinition(swAngleMateData, swAssy, Nothing)
```

```
End If
```

```
End Sub
```

Create and Edit Limit Distance Mate Example (C#)

This example shows how to create and edit a limit distance mate.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo1.sldasm
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a limit distance mate.
// 2. Press F5 to continue.
// 3. Modifies the mate alignment of the limit distance mate.
// 4. Inspect the Immediate window and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace LimitDistanceMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private AssemblyDoc swAssy;
        private DistanceMateFeatureData swDistMateData;
        private MateFeatureData swMateData;
        private SelectionMgr swSelMgr;
        private bool boolstatus;
        private Feature feat;
        private Face2[] facesDist = new Face2[2];
        private object vFacesDist;
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            swMateData = (MateFeatureData)swAssy.CreateMateData(5); //Distance mate

            Debug.Print("Type of mate as defined in swMateType_e: " + swMateData.TypeName);

            swDistMateData = (DistanceMateFeatureData)swMateData;
            swDistMateData.MateAlignment = (int)swMateAlign_e.swMateAlignALIGNED;
            swDistMateData.FlipDimension = false;
            swDistMateData.Distance = 0.0254;
            swDistMateData.MaximumDistance = 0.0254;
            swDistMateData.MinimumDistance = 0.01;

            boolstatus = swModel.Extension.SelectByRay(0.00868857956595548, 0.0414144214960288, 0.0633435410960033, -0.5201487747284
            boolstatus = swModel.Extension.SelectByRay(0.0313565896258297, 0.0296508617577445, 0.0442099188854286, 0.340524666870961

            facesDist[0] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            facesDist[1] = (Face2)swSelMgr.GetSelectedObject6(2, -1);

            vFacesDist = facesDist;

            swDistMateData.EntitiesToMate = vFacesDist;

            feat = (Feature)swAssy.CreateMate(swDistMateData);

            swModel.GraphicsRedraw2();

            System.Diagnostics.Debugger.Break();

            feat = swModel.Extension.GetLastFeatureAdded();
            Debug.Print("Feature GetTypeName2 of mate created is " + feat.GetTypeName2());

            swMateData = (MateFeatureData)feat.GetDefinition();

            if (swMateData.TypeName == (int)swMateType_e.swMateDISTANCE)
            {
                swDistMateData = (DistanceMateFeatureData)swMateData;
                Debug.Print("Distance mate alignment is " + swDistMateData.MateAlignment);

                if (swDistMateData.MateAlignment == (int)swMateAlign_e.swMateAlignALIGNED)
                {
                    swDistMateData.MateAlignment = (int)swMateAlign_e.swMateAlignANTI_ALIGNED;
                }
                else
            }
        }
    }
}
```

```

    {
        swDistMateData.MateAlignment = (int)swMateAlign_e.swMateAlignALIGNED;
    }

    Debug.Print("Distance mate alignment changed to " + swDistMateData.MateAlignment);
    Debug.Print("Distance: " + swDistMateData.Distance);
    Debug.Print("Minimum distance: " + swDistMateData.MinimumDistance);
    Debug.Print("Maximum distance: " + swDistMateData.MaximumDistance);
    Debug.Print("Move entities to opposite sides of the dimension? " + swDistMateData.FlipDimension);

    boolstatus = feat.ModifyDefinition(swDistMateData, swAssy, null);
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Create and Edit Limit Distance Mate Example (VBA)

This example shows how to create and edit a limit distance mate.

```

'-----
' Preconditions:
' 1. Open:
'   public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo1.sldasm
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a limit distance mate.
' 2. Press F5 to continue.
' 3. Modifies the mate alignment of the limit distance mate.
' 4. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swDistMateData As SldWorks.DistanceMateFeatureData
Dim swMateData As SldWorks.MateFeatureData
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim feat As SldWorks.Feature
Dim facesDist(1) As SldWorks.Face2
Dim vFacesDist As Variant

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager

    Set swMateData = swAssy.CreateMateData(5) 'Distance mate

    Debug.Print "Type of mate as defined in swMateType_e: " & swMateData.TypeName

    Set swDistMateData = swMateData
    swDistMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED
    swDistMateData.FlipDimension = False
    swDistMateData.Distance = 0.0254
    swDistMateData.MaximumDistance = 0.0254
    swDistMateData.MinimumDistance = 0.01

    boolstatus = swModel.Extension.SelectByRay(8.68857956595548E-03, 4.14144214960288E-02, 6.33435410960033E-02,
-0.520148774728431, -0.59141018013918, -0.616181183562315, 4.68381592786756E-04, 2, True, 1, 0)
    boolstatus = swModel.Extension.SelectByRay(3.13565896258297E-02, 2.96508617577445E-02, 4.42099188854286E-02, 0.340524666870961,
-0.380278973953885, -0.859901653226112, 4.31713609895031E-04, 2, True, 1, 0)

    Set facesDist(0) = swSelMgr.GetSelectedObject6(1, -1)
    Set facesDist(1) = swSelMgr.GetSelectedObject6(2, -1)

    vFacesDist = facesDist

    swDistMateData.EntitiesToMate = vFacesDist

    Set feat = swAssy.CreateMate(swDistMateData)

    swModel.GraphicsRedraw2

    Stop

    Set feat = swModel.Extension.GetLastFeatureAdded
    Debug.Print "Feature GetTypeName2 of mate created is " & feat.GetTypeName2

    Set swMateData = feat.GetDefinition

    If (swMateData.TypeName = SwConst.swMateType_e.swMateDISTANCE) Then

        Set swDistMateData = swMateData
        Debug.Print "Distance mate alignment is " & swDistMateData.MateAlignment

        If (swDistMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED) Then
            swDistMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignANTI_ALIGNED
        Else
            swDistMateData.MateAlignment = SwConst.swMateAlign_e.swMateAlignALIGNED
        End If
    End If
End Sub

```

End If

```
Debug.Print "Distance mate alignment changed to " & swDistMateData.MateAlignment  
Debug.Print "Distance: " & swDistMateData.Distance  
Debug.Print "Minimum distance: " & swDistMateData.MinimumDistance  
Debug.Print "Maximum distance: " & swDistMateData.MaximumDistance  
Debug.Print "Move entities to opposite sides of the dimension? " & swDistMateData.FlipDimension
```

```
boolstatus = feat.ModifyDefinition(swDistMateData, swAssy, Nothing)
```

End If

End Sub

Create Line From User Selection Example (VBA)

This example shows how to create geometry in a sketch or drawing, including how to retrieve the coordinates of a user's selection.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Click anywhere on the model.  
'  
' Postconditions:  
' 1. Creates a line starting at the selected point in model  
'    space coordinates from the currently selected object  
'    and ending at the specified coordinates.  
' 2. Examine the graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim PickPt As Variant  
Dim x1, y1, x2, y2 As Double  
  
Sub main()  
  
    Set swApp = CreateObject("SldWorks.Application")  
  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    ' Get the selection point coordinates in model space  
    ' from the currently selected object  
    PickPt = swSelMgr.GetSelectionPointInSketchSpace2(1, -1)  
  
    ' If no selection was made, then return an error message  
    If (swSelMgr.GetSelectedObjectCount = 0) Or (IsEmpty(PickPt)) Then  
        swApp.SendMsgToUser ("Click anywhere on the model, then rerun the macro.")  
    ' Otherwise, create a line  
    Else  
        ' Set line start point  
        x1 = PickPt(0)  
        y1 = PickPt(1)  
        ' Set line end point  
        x2 = PickPt(0) + 0.5  
        y2 = PickPt(1) + 0.5  
        ' Create the line  
        swModel.CreateLineVB x1, y1, 0#, x2, y2, 0#  
    End If  
  
End Sub
```

Create and Edit Linear-Linear Coupler Mate Example (C#)

This example shows how to create and edit a linear/linear coupler mate.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo4.sldasm
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a linear/linear coupler mate.
// 2. Press F5 to continue.
// 3. Changes the reverse property of the mate.
// 4. Inspect the Immediate window and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace LinearCouplerMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private AssemblyDoc swAssy;
        private LinearCouplerMateFeatureData swLinearMateData;
        private MateFeatureData swMateData;
        private SelectionMgr swSelMgr;
        private bool boolstatus;
        private Feature feat;
        private Face2[] facesLC = new Face2[2];
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            swMateData = (MateFeatureData)swAssy.CreateMateData(18); //Linear/linear coupler mate

            if (swMateData.TypeName == (int)swMateType_e.swMateLINEARCOUPLER)
            {
                swLinearMateData = (LinearCouplerMateFeatureData)swMateData;

                swLinearMateData.CouplerRatioNumerator = 0.00099;
                Debug.Print("Linear/linear coupler ratio numerator is " + swLinearMateData.CouplerRatioNumerator);

                swLinearMateData.CouplerRatioDenominator = 0.00199;
                Debug.Print("Linear/linear coupler ratio denominator is " + swLinearMateData.CouplerRatioDenominator);

                swLinearMateData.Reverse = false;

                boolstatus = swModel.Extension.SelectByRay(-0.0805427342211829, 0.00068649651763053, 0.0119585811536354, 0.722099295);
                boolstatus = swModel.Extension.SelectByRay(-0.0435951139756412, -0.00915195091067744, 0.0234497167883774, 0.722099295);

                facesLC[0] = (Face2)swSelMgr.GetSelectedObject6(1, -1);
                facesLC[1] = (Face2)swSelMgr.GetSelectedObject6(2, -1);

                object vFacesLC = facesLC;

                swLinearMateData.MateEntity1 = facesLC[0];
                swLinearMateData.MateEntity2 = facesLC[1];

                feat = (Feature)swAssy.CreateMate(swLinearMateData);
            }

            swModel.GraphicsRedraw2();

            System.Diagnostics.Debugger.Break();

            feat = swModel.Extension.GetLastFeatureAdded();

            Debug.Print("Feature GetTypeName2 of mate created is " + feat.GetTypeName2());

            swMateData = (MateFeatureData)feat.GetDefinition();

            if (swMateData.TypeName == (int)swMateType_e.swMateLINEARCOUPLER)
            {

```

```

swLinearMateData = (LinearCouplerMateFeatureData)swMateData;

Debug.Print("Linear/linear coupler mate reverse is " + swLinearMateData.Reverse);

if (swLinearMateData.Reverse == true)
{
    swLinearMateData.Reverse = false;
}
else
{
    swLinearMateData.Reverse = true;
}

Debug.Print("Linear/linear coupler mate reverse changed to " + swLinearMateData.Reverse);

boolstatus = feat.ModifyDefinition(swLinearMateData, swAssy, null);

    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Create and Edit Linear-Linear Coupler Mate Example (VBA)

This example shows how to create and edit a linear/linear coupler mate.

```

'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\AdvancedMates\AdvancedMateDemo4.sldasm
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a linear/linear coupler mate.
' 2. Press F5 to continue.
' 3. Changes the reverse property of the mate.
' 4. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swLinearMateData As SldWorks.LinearCouplerMateFeatureData
Dim swMateData As SldWorks.MateFeatureData
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Dim feat As SldWorks.Feature
Dim facesLC(1) As SldWorks.Face2

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager

    Set swMateData = swAssy.CreateMateData(18) 'Linear/linear coupler mate

    If (swMateData.TypeName = SwConst.swMateType_e.swMateLINEARCOUPLER) Then

        Set swLinearMateData = swMateData

        swLinearMateData.CouplerRatioNumerator = 0.00099
        Debug.Print "Linear/linear coupler ratio numerator is " & swLinearMateData.CouplerRatioNumerator

        swLinearMateData.CouplerRatioDenominator = 0.00199
        Debug.Print "Linear/linear coupler ratio denominator is " & swLinearMateData.CouplerRatioDenominator

        swLinearMateData.Reverse = False

        boolstatus = swModel.Extension.SelectByRay(-8.05427342211829E-02, 6.8649651763053E-04, 1.19585811536354E-02,
0.722099295067451, -0.530192163929043, -0.44437470379324, 1.35613437918167E-03, 2, False, 0, 0)
        boolstatus = swModel.Extension.SelectByRay(-4.35951139756412E-02, -9.15195091067744E-03, 2.34497167883774E-02,
0.722099295067451, -0.530192163929043, -0.44437470379324, 1.35613437918167E-03, 2, True, 0, 0)

        Set facesLC(0) = swSelMgr.GetSelectedObject6(1, -1)
        Set facesLC(1) = swSelMgr.GetSelectedObject6(2, -1)

        Dim vFacesLC As Variant
        vFacesLC = facesLC

        swLinearMateData.MateEntity1 = facesLC(0)
        swLinearMateData.MateEntity2 = facesLC(1)

        Set feat = swAssy.CreateMate(swLinearMateData)

    End If

    swModel.GraphicsRedraw2

    Stop

    Set feat = swModel.Extension.GetLastFeatureAdded

    Debug.Print "Feature GetTypeName2 of mate created is " & feat.GetTypeName2

    Set swMateData = feat.GetDefinition

```

```
If (swMateData.TypeName = SwConst.swMateType_e.swMateLINEARCOUPLER) Then

    Set swLinearMateData = swMateData

    Debug.Print "Linear/linear coupler mate reverse is " & swLinearMateData.Reverse

    If (swLinearMateData.Reverse = True) Then
        swLinearMateData.Reverse = False
    Else
        swLinearMateData.Reverse = True
    End If

    Debug.Print "Linear/linear coupler mate reverse changed to " & swLinearMateData.Reverse

    boolstatus = feat.ModifyDefinition(swLinearMateData, swAssy, Nothing)

End If
End Sub
```

Create Linear Motor Feature Example (VBA)

This example shows how to create a motor feature data object and a linear motor feature.

```

'-----
' Postconditions:
' 1. Open an assembly document that contains at least
'    two components and a motion study named Motion Study 3.
' 2. In the IDE, add a reference to the SolidWorks MotionStudy
'    type library.
' 3. In SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Motion > OK.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Gets the MotionManager.
' 2. Gets and activates a motion study named Motion Study 3.
' 3. Selects a face on a component.
' 4. Creates and sets data for a linear motor feature.
' 5. Selects the component whose face was selected in step 3.
' 6. Selects a face on a different component.
' 7. Sets motion relative to the first selected component.
' 8. Select the same face on the first component again and
'    set that face to be the load-bearing face for the motor feature.
' 9. Creates a linear motor feature.
' 10. Gets the name of linear motor feature.
' 11. Examine the Immediate window.
'
' NOTE: For this example to work for you, replace calls to
' select the different faces and components with calls to select the faces
' and components on your assembly. You must also create a file called
' Test_bouncingBall.csv that contains the spline data, and your assembly
' must have a motion-analysis motion study named Motion Study 3.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swMotionMgr As SwMotionStudy.MotionStudyManager
    Dim swMotionStudy3 As SwMotionStudy.MotionStudy
    Dim swMotorFeat As SldWorks.SimulationMotorFeatureData
    Dim swGravityFeat As Object
    Dim boolstatus As Boolean
    Dim swFeat As SldWorks.Feature

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Get the MotionManager
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    If (swMotionMgr Is Nothing) Then
        End
    End If

    ' Get Motion Study 3, a Motion Analysis study
    Set swMotionStudy3 = swMotionMgr.GetMotionStudy("Motion Study 3")

    If (swMotionStudy3 Is Nothing) Then
        MsgBox "Motion Study 3 is not available."
        End
    End If

    ' Active Motion Study 3
    swMotionStudy3.Activate

    ' Select a face on a component
    Set swSelMgr = swModel.SelectionManager
    boolstatus = swModelDocExt.SelectByID("", "FACE", -0.07792618280496, 0.06212618843159, 0.02214691016243, False, 0, Nothing, 0)

    ' Create linear motor feature data
    ' and get and set some options
    Set swMotorFeat = swMotionStudy3.CreateDefinition(swFmAEMLinearMotor)
    If swMotorFeat Is Nothing Then
        Debug.Print "ERROR: Creation of motor feature data object failed."
        Exit Sub
    End If
    swMotorFeat.InterpolatedMotor swSimulationMotorDrive_Velocity, 1
    swMotorFeat.DirectionReference = swSelMgr.GetSelectedObject6(1, -1)
    ' Load the spline data from the specified file
    boolstatus = swMotorFeat.LoadSplineData("Test_bouncingBall.csv")

    ' Select the component on whose face you previously selected
    Dim RelObj As SldWorks.Component2
    Set RelObj = swSelMgr.GetSelectedObjectsComponent3(1, -1)

    ' Select a face on a different component
    boolstatus = swModelDocExt.SelectByID("", "FACE", -0.07924982844941, 0.06212618843165, 0.03225592518596, False, 0, Nothing, 0)

    ' Set motion relative to the first selected component
    swMotorFeat.RelativeComponent = RelObj
    swMotorFeat.Location = swSelMgr.GetSelectedObject6(1, -1)

```

```

' Select the same face on the first component again and
' Set that face to be the load-bearing face for the motor feature
boolstatus = swModelDocExt.SelectByID("", "FACE", -0.07792618280496, 0.06212618843159, 0.02214691016243, False, 0, Nothing, 0)
Dim ContactObj(0) As Object
Set ContactObj(0) = swSelMgr.GetSelectedObject6(1, -1)
Dim vContact As Variant
vContact = ContactObj
swMotorFeat.LoadReferences = vContact

' Get the motor type
Debug.Print swMotorFeat.MotorType

' Create a motor feature
Set swFeat = swMotionStudy3.CreateFeature(swMotorFeat)

If swFeat Is Nothing Then
    Debug.Print " ERROR: Creation of the motor feature failed."
Else
    Debug.Print "Name of the feature added : " & swFeat.Name
End If

End Sub

```

Create Linear Pattern Example (C#)

This example shows how to create a linear-pattern feature using an offset reference.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\varyinstance.sldprt.
//
// Postconditions: Creates LPattern1 in the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save any changes.
//-----

using

Microsoft.VisualBasic;
using

System;
using

System.Collections;
using

System.Collections.Generic;
using

System.Data;
using

System.Diagnostics;
using

SolidWorks.Interop.sldworks;
using

SolidWorks.Interop.swconst;
using

System.Runtime.InteropServices;
namespace
FeatureLinearPattern4_CSharp.csproj
{

    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;
        bool boolstatus;
        public void Main()
        {
            Part = (
                ModelDoc2)swApp.ActiveDoc;

            // Select feature to pattern
            boolstatus = Part.Extension.SelectByID(
                "Cut-Extrude1", "BODYFEATURE", 0, 0, 0, false, 4, null, (int)swSelectOption_e.swSelectOptionDefault);

            // Select Direction 1 reference
            boolstatus = Part.Extension.SelectByID(
                "", "EDGE", -0.000143804142453519, 0.0690197499537817, 0.0370101655861568, true, 1, null, (int)swSelectOption_e.swSelectOptionDefault);

            // Select Direction 2 reference
            boolstatus = Part.Extension.SelectByID(
                "", "EDGE", 8.50674319963218E-06, -0.0507775663234327, 0.14947002782122, true, 2, null, (int)swSelectOption_e.swSelectOptionDefault);

            // Select face from which to offset the pattern's furthest instance in Direction 1
            boolstatus = Part.Extension.SelectByID(
                "", "FACE", 0.00799245561853468, -0.0329718247828055, -0.0491565136701766, true, 2097152, null, (int)swSelectOption_e.swSelectOptionDefault);

            // Select seed instance edge with which to measure distances between pattern instances
            boolstatus = Part.Extension.SelectByID(
                "", "EDGE", 0.0114738185224041, 0.00104517477574007, -0.0334013867415592, true, 8388608, null, (int)swSelectOption_e.swSelectOptionDefault);

            // Create linear-pattern feature calling IFeatureManager::FeatureLinearPattern4
            // with the following parameter information:
            //
            // * 3 (Num1)
            // * Spacing1 (not used)
            // * 4 (Num2)
            // * 0.02 (Spacing2 in mm)
            // * True (FlipDir1)
            // * True (Flipdir2)
            // * DName1 (not used)
            // * DName2 (not used)
            // * False (GeometryPattern)
```

```

// * False (VaryInstance)
// * True (HasOffset1)
// * HasOffset2 (not used)
// * True (CtrlByNum1; control pattern instance spacing in Direction 1
//     using number of instances instead of spacing)
// * CtrlByNum2(not used)
// * False (FromCentroid1; measure distances between pattern instances in
//     Direction 1 using the selected pattern seed edge marked with
//     8388608)
// * FromCentroid2 (not used)
// * False (RevOffset1; do not reverse Offset1)
// * False (RevOffset2; do not reverse Offset2)
// * 0.19 (Offset1; furthest instance in Direction 1 is offset by 190 mm
//     from the selected face marked with 2097152)
// * Offset2 (not used)
//
myFeature = Part.FeatureManager.FeatureLinearPattern4(3, 0.0029375, 4, 0.02,
true, true, "", "", false, false,
true, false, true, false, false, true, false, false, 0.19, 0.01);
}

public SldWorks swApp;
}
}

```

Create Linear Pattern Example (VBA)

This example shows how to create a linear-pattern feature using an offset reference.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\varyinstance.sldprt.
'
' Postconditions: Creates LPattern1 in the FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save any changes.
'-----
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myFeature As SldWorks.Feature
Dim boolstatus As Boolean
Option Explicit

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

    ' Select feature to pattern
    boolstatus = Part.Extension.SelectByID2("Cut-Extrude1", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, swSelectOptionDefault)

    ' Select Direction 1 reference
    boolstatus = Part.Extension.SelectByID2("", "EDGE", -1.43804142453519E-04, 6.90197499537817E-02, 3.70101655861568E-02, True, 1, Nothing, swSelectOptionDefault)

    ' Select Direction 2 reference
    boolstatus = Part.Extension.SelectByID2("", "EDGE", 8.50674319963218E-06, -5.07775663234327E-02, 0.14947002782122, True, 2, Nothing, swSelectOptionDefault)

    ' Select face from which to offset the pattern's furthest instance in Direction 1
    boolstatus = Part.Extension.SelectByID2("", "FACE", 7.99245561853468E-03, -3.29718247828055E-02, -4.91565136701766E-02, True, 2097152, Nothing, swSelectOptionDefault)

    ' Select seed instance edge with which to measure distances between pattern instances
    boolstatus = Part.Extension.SelectByID2("", "EDGE", 1.14738185224041E-02, 1.04517477574007E-03, -3.34013867415592E-02, True, 8388608, Nothing, swSelectOptionDefault)

    ' Create linear-pattern feature calling IFeatureManager::FeatureLinearPattern4
    ' with the following parameter information:
    '
    ' * 3 (Num1)
    ' * Spacing1 (not used)
    ' * 4 (Num2)
    ' * 0.02 (Spacing2 in mm)
    ' * True (FlipDir1)
    ' * True (FlipDir2)
    ' * DName1 (not used)
    ' * DName2 (not used)
    ' * False (GeometryPattern)
    ' * False (VaryInstance)
    ' * True (hasOffset1)
    ' * HasOffset2 (not used)
    ' * True (CtrlByNum1; control pattern instance spacing in Direction 1
    '   using number of instances instead of spacing)
    ' * CtrlByNum2(not used)
    ' * False (FromCentroid1; measure distances between pattern instances in
    '   Direction 1 using the selected pattern seed edge marked with
    '   8388608)
    ' * FromCentroid2 (not used)
    ' * False (RevOffset1; do not reverse Offset1)
    ' * False (RevOffset2; do not reverse Offset2)
    ' * 0.19 (Offset1; furthest instance in Direction 1 is offset by 190 mm
    '   from the selected face marked with 2097152)
    ' * Offset2 (not used)
    '
    Set myFeature = Part.FeatureManager.FeatureLinearPattern4( _
    3, _
    0.0029375, _
    4, _
    0.02, _
    True, _
    True, _
    "", _
    "", _
    False, _
    False, _
    True, _
    False, _
    True, _
    False, _
    False, _
    True, _
    False, _
    False, _
    True, _
    False, _
    )

```

```
False, _  
0.19, _  
0.01 _  
)  
End Sub
```

Create Linear Pattern Example (VB.NET)

This example shows how to create a linear-pattern feature using an offset reference.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\varyinstance.sldprt.
'
' Postconditions: Creates LPattern1 in the FeatureManager design tree.
'
' NOTE: Because the model is used elsewhere, do not save any changes.
'-----

```

Imports

SolidWorks.Interop.sldworks

Imports

SolidWorks.Interop.swconst

Imports

System.Runtime.InteropServices

Imports

System

Partial

Class SolidWorksMacro

Dim Part As ModelDoc2

Dim myFeature As Feature

Dim boolstatus As Boolean

Sub main()

Part = swApp.ActiveDoc

' Select feature to pattern

boolstatus = Part.Extension.SelectByID2(

"Cut-Extrude1", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, swSelectOption_e.swSelectOptionDefault)

' Select Direction 1 reference

boolstatus = Part.Extension.SelectByID2(

"" , "EDGE", -0.000143804142453519, 0.0690197499537817, 0.0370101655861568, True, 1, Nothing, swSelectOption_e.swSelectOptionDefault)

' Select Direction 2 reference

boolstatus = Part.Extension.SelectByID2(

"" , "EDGE", 0.00000850674319963218, -0.0507775663234327, 0.14947002782122, True, 2, Nothing, swSelectOption_e.swSelectOptionDefault)

' Select face from which to offset the pattern's furthest instance in Direction 1

boolstatus = Part.Extension.SelectByID2(

"" , "FACE", 0.00799245561853468, -0.0329718247828055, -0.0491565136701766, True, 2097152, Nothing, swSelectOption_e.swSelectOptionDefault)

' Select seed instance edge with which to measure distances between pattern instances

boolstatus = Part.Extension.SelectByID2(

"" , "EDGE", 0.0114738185224041, 0.00104517477574007, -0.0334013867415592, True, 8388608, Nothing, swSelectOption_e.swSelectOptionDefault)

' Create linear-pattern feature calling IFeatureManager::FeatureLinearPattern4

' with the following parameter information:

'

' * 3 (Num1)

' * Spacing1 (not used)

' * 4 (Num2)

' * 0.02 (Spacing2 in mm)

' * True (FlipDir1)

' * True (FlipDir2)

' * DName1 (not used)

' * DName2 (not used)

' * False (GeometryPattern)

' * False (VaryInstance)

' * True (HasOffset1)

' * HasOffset2 (not used)

' * True (CtrlByNum1; control pattern instance spacing in Direction 1

using number of instances instead of spacing)

' * CtrlByNum2(not used)

' * False (FromCentroid1; measure distances between pattern instances in

Direction 1 using the selected pattern seed edge marked with

8388608)

' * FromCentroid2 (not used)

' * False (RevOffset1; do not reverse Offset1)

' * False (RevOffset2; do not reverse Offset2)

```

' * 0.19 (Offset1; furthest instance in Direction 1 is offset by 190 mm
'      from the selected face marked with 2097152)
' * Offset2 (not used)
'
myFeature = Part.FeatureManager.FeatureLinearPattern4( _
3, _
0.0029375, _
4, _
0.02, _
True, _
True, _
"" _
"" , _
False, _
False, _
True, _
False, _
True, _
False, _
False, _
True, _
False, _
False, _
0.19, _
0.01 _
)
End Sub

Public swApp As SldWorks

End
Class

```

Create Linear Pattern of Subassembly Example (C#)

This example shows how to create a linear pattern of a subassembly.

```
//-----
// Preconditions: Verify that the assembly exists.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects an edge for Direction 1.
// 3. Selects a subassembly to pattern.
// 4. Creates a linear pattern.
// 5. Examine the FeatureManager design tree and
//    graphics area.
//
// NOTE: Because the assembly is used elsewhere, do
// not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\distance linkage.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            status = swModelDocExt.SelectByID2("", "EDGE", 0.222723097227572, -0.193386735236572, -0.10172211746567, false, 2, null, 0);
            status = swModelDocExt.SelectByID2("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, true, 1, null, 0);
            swFeature = (Feature)swFeatureManager.FeatureLinearPattern5(4, 0.254, 1, 0.05, false, false, "NULL", "NULL", false, false,
            false, false, false, true, true, false, false, 0, 0, false,
            false);
            swModel.ClearSelection2(true);
            swModel.ViewZoomtofit2();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Linear Pattern of Subassembly Example (VBA)

This example shows how to create a linear pattern of a subassembly.

' Preconditions: Verify that the assembly exists.
,

' Postconditions:
' 1. Opens the assembly.
' 2. Selects an edge for Direction 1.
' 3. Selects a subassembly to pattern.
' 4. Creates a linear pattern.
' 5. Examine the FeatureManager design tree and
 graphics area.
,

' NOTE: Because the assembly is used elsewhere, do
' not save changes.

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
```

Sub main()

```
    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\distance linkage.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager
    status = swModelDocExt.SelectByID("", "EDGE", 0.222723097227572, -0.193386735236572, -0.10172211746567, False, 2, Nothing, 0)
    status = swModelDocExt.SelectByID("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
    Set swFeature = swFeatureManager.FeatureLinearPattern5(4, 0.254, 1, 0.05, False, False, "NULL", "NULL", False, False, False, False, False, False,
True, True, False, False, 0, 0, False, False)
    swModel.ClearSelection2 True
    swModel.ViewZoomtofit2
```

End Sub

Create Linear Pattern of Subassembly Example_VBNET (Topic)

SOLIDWORKS API Help

Create Linear Pattern of Subassembly Example (VB.NET)

This example shows how to create a linear pattern of a subassembly.

```
-----
' Preconditions: Verify that the assembly exists.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Selects an edge for Direction 1.
' 3. Selects a subassembly to pattern.
' 4. Creates a linear pattern.
' 5. Examine the FeatureManager design tree and
'    graphics area.
'
' NOTE: Because the assembly is used elsewhere, do
' not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\distance linkage.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager
        status = swModelDocExt.SelectByID("", "EDGE", 0.222723097227572, -0.193386735236572, -0.10172211746567, False, 2, Nothing, 0)
        status = swModelDocExt.SelectByID("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
        swFeature = swFeatureManager.FeatureLinearPattern5(4, 0.254, 1, 0.05, False, False, "NULL", "NULL", False, False, False, False, False, True
, True, False, False, 0, 0, False, False)
        swModel.ClearSelection2(True)
        swModel.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Create Local Circular Pattern Example (C#)

This example shows how to create a local circular pattern feature.

```
//-----
// Preconditions: Verify that the assembly exists.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects an edge for the direction axis.
// 3. Selects a subassembly to pattern.
// 4. Creates LocalCircularPattern1.
// 5. Examine the FeatureManager design tree and graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace CreateLocalCircularPattern_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private ModelDocExtension swModelDocExt;
        private FeatureManager swFeatureManager;
        private Feature swFeature;
        private LocalCircularPatternFeatureData swLocalCirPattFD;
        private bool status;
        private int errors = 0;
        private int warnings = 0;
        private string fileName;
        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\distance linkage.sldasm";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent);
            swModelDocExt = swModel.Extension;
            swFeatureManager = swModel.FeatureManager;
            status = swModelDocExt.SelectByID("", "EDGE", 0.22639417933982, -0.194822643434378, 0.102086175644843, false, 2, null,
            status = swModelDocExt.SelectByID("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, true, 1, null, 0);
            swLocalCirPattFD = (LocalCircularPatternFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmLocalCi
            swLocalCirPattFD.TotalInstances = 3;
            swLocalCirPattFD.EqualSpacing = true;
            swFeature = swFeatureManager.CreateFeature(swLocalCirPattFD);
            swModel.ClearSelection2(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Local Curve-Driven Pattern Example (C#)

This example shows how to create a local curve-driven pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly document.
// 2. Selects the component and edge for the
//    local curve-driven pattern feature.
// 3. Creates a local curve-driven pattern feature.
// 4. Gets values and settings of the local curve-driven
//    pattern feature.
// 5. Examine the Immediate window and graphics area.
//
// NOTE: Because this assembly is used elsewhere, do not save
// changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FeatureLocalCurveDrivenCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            LocalCurvePatternFeatureData swLocalCurvePatternFeat = default(LocalCurvePatternFeatureData);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            //Open assembly document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\assem20.sldasm";

            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            //Select a component and an edge
            //for the local curve-driven pattern
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("block20-1@assem20", "COMPONENT", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.131144367102934, 0.0844607238593085, -0.124705856534547, true, 2, null, 0);

            //Create local curve-driven pattern feature
            swFeatMgr = (FeatureManager)swModel.FeatureManager;

            swLocalCurvePatternFeat = (LocalCurvePatternFeatureData)swFeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmLocalCurvePattern);
            swLocalCurvePatternFeat.D1AlignmentMethod = 1;
            swLocalCurvePatternFeat.D1CurveMethod = 1;
            swLocalCurvePatternFeat.D1InstanceCount = 3;
            swLocalCurvePatternFeat.D1IsEqualSpaced = true;
            swLocalCurvePatternFeat.D1ReferencePoint = 1;
            swLocalCurvePatternFeat.D1ReverseDirection = false;
            swLocalCurvePatternFeat.D1Spacing = 0.05;
            swLocalCurvePatternFeat.D2InstanceCount = 1;
            swLocalCurvePatternFeat.D2IsEqualSpaced = false;
            swLocalCurvePatternFeat.D2PatternSeedOnly = false;
            swLocalCurvePatternFeat.D2ReverseDirection = false;
            swLocalCurvePatternFeat.D2Spacing = 0.05;
            swLocalCurvePatternFeat.Dir2Specified = false;
            swFeat = (Feature)swFeatMgr.CreateFeature(swLocalCurvePatternFeat);

            //Get local curve-driven pattern feature data
            swLocalCurvePatternFeat = (LocalCurvePatternFeatureData)swFeat.GetDefinition();
            Debug.Print("Local curve-driven pattern properties: ");
            Debug.Print(" Number of components: " + swLocalCurvePatternFeat.GetPatternComponentCount());
            Debug.Print(" Number of skipped items: " + swLocalCurvePatternFeat.GetSkippedItemCount());
            Debug.Print(" Direction 1: ");
            Debug.Print(" Number of pattern instances: " + swLocalCurvePatternFeat.D1InstanceCount);
            Debug.Print(" Is direction flipped: " + swLocalCurvePatternFeat.D1ReverseDirection);
            Debug.Print(" Pattern instances equal spaced: " + swLocalCurvePatternFeat.D1IsEqualSpaced);
            Debug.Print(" Spacing for pattern instances: " + swLocalCurvePatternFeat.D1Spacing);
            Debug.Print(" Curve method: " + swLocalCurvePatternFeat.D1CurveMethod);
            Debug.Print(" Alignment method: " + swLocalCurvePatternFeat.D1AlignmentMethod);
            Debug.Print(" Direction 2:");
            Debug.Print(" Is Direction 2 specified: " + swLocalCurvePatternFeat.Dir2Specified);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Local Curve-driven Pattern Example (VBA)

This example shows how to create a local curve-driven pattern feature.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly document.
' 2. Selects the component and edge for the
'    local curve-driven pattern feature.
' 3. Creates a local curve-driven pattern feature.
' 4. Gets values and settings of the local curve-driven
'    pattern feature.
' 5. Examine the Immediate window and graphics area.
'
' NOTE: Because this assembly is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swLocalCurvePatternFeat As SldWorks.LocalCurvePatternFeatureData
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\assem20.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select a component and an edge
    'for the local curve-driven pattern
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("block20-1@assem20", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "EDGE", 0.131144367102934, 8.44607238593085E-02, -0.124705856534547, True, 2, Nothing, 0)

    'Create local curve-driven pattern feature
    Set swFeatMgr = swModel.FeatureManager
    Set swLocalCurvePatternFeat = swFeatMgr.CreateDefinition(swFmLocalCurvePattern)
    swLocalCurvePatternFeat.D1AlignmentMethod = 1
    swLocalCurvePatternFeat.D1CurveMethod = 1
    swLocalCurvePatternFeat.D1InstanceCount = 3
    swLocalCurvePatternFeat.D1IsEqualSpaced = True
    swLocalCurvePatternFeat.D1ReferencePoint = 1
    swLocalCurvePatternFeat.D1ReverseDirection = False
    swLocalCurvePatternFeat.D1Spacing = 0.05
    swLocalCurvePatternFeat.D2InstanceCount = 1
    swLocalCurvePatternFeat.D2IsEqualSpaced = False
    swLocalCurvePatternFeat.D2PatternSeedOnly = False
    swLocalCurvePatternFeat.D2ReverseDirection = False
    swLocalCurvePatternFeat.D2Spacing = 0.05
    swLocalCurvePatternFeat.Dir2Specified = False
    Set swFeat = swFeatMgr.CreateFeature(swLocalCurvePatternFeat)

    'Get local curve-driven pattern feature data
    Set swLocalCurvePatternFeat = swFeat.GetDefinition
    Debug.Print "Local curve-driven pattern properties: "
    Debug.Print "  Number of components: " & swLocalCurvePatternFeat.GetPatternComponentCount
    Debug.Print "  Number of skipped items: " & swLocalCurvePatternFeat.GetSkippedItemCount
    Debug.Print "  Direction 1: "
    Debug.Print "    Number of pattern instances: " & swLocalCurvePatternFeat.D1InstanceCount
    Debug.Print "    Is direction flipped: " & swLocalCurvePatternFeat.D1ReverseDirection
    Debug.Print "    Pattern instances equal spaced: " & swLocalCurvePatternFeat.D1IsEqualSpaced
    Debug.Print "    Spacing for pattern instances: " & swLocalCurvePatternFeat.D1Spacing
    Debug.Print "    Curve method: " & swLocalCurvePatternFeat.D1CurveMethod
    Debug.Print "    Alignment method: " & swLocalCurvePatternFeat.D1AlignmentMethod
    Debug.Print "  Direction 2:"
    Debug.Print "    Is Direction 2 specified: " & swLocalCurvePatternFeat.Dir2Specified

```

End Sub

Create Local Curve-Driven Pattern Example (VB.NET)

This example shows how to create a local curve-driven pattern feature.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly document.
' 2. Selects the component and edge for the
'    local curve-driven pattern feature.
' 3. Creates a local curve-driven pattern feature.
' 4. Gets values and settings of the local curve-driven
'    pattern feature.
' 5. Examine the Immediate window and graphics area.
'
' NOTE: Because this assembly is used elsewhere, do not save
' changes.
'-----

```

```

Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst

```

Partial Class SolidWorksMacro

Public Sub Main()

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swFeatMgr As FeatureManager
    Dim swFeat As Feature
    Dim swLocalCurvePatternFeat As LocalCurvePatternFeatureData
    Dim fileName As String
    Dim status As Boolean
    Dim errors As Integer
    Dim warnings As Integer

    'Open assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\assem20.sldasm"
    swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,

    'Select a component and an edge
    'for the local curve-driven pattern
    swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("block20-1@assem20", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "EDGE", 0.131144367102934, 0.08444607238593085, -0.124705856534547, True, 2, Nothing,

    'Create local curve-driven pattern feature
    swFeatMgr = swModel.FeatureManager

    swLocalCurvePatternFeat = swFeatMgr.CreateDefinition(swFeatureNameID_e.swFmLocalCurvePattern)
    swLocalCurvePatternFeat.D1AlignmentMethod = 1
    swLocalCurvePatternFeat.D1CurveMethod = 1
    swLocalCurvePatternFeat.D1InstanceCount = 3
    swLocalCurvePatternFeat.D1IsEqualSpaced = True
    swLocalCurvePatternFeat.D1ReferencePoint = 1
    swLocalCurvePatternFeat.D1ReverseDirection = False
    swLocalCurvePatternFeat.D1Spacing = 0.05
    swLocalCurvePatternFeat.D2InstanceCount = 1
    swLocalCurvePatternFeat.D2IsEqualSpaced = False
    swLocalCurvePatternFeat.D2PatternSeedOnly = False
    swLocalCurvePatternFeat.D2ReverseDirection = False
    swLocalCurvePatternFeat.D2Spacing = 0.05
    swLocalCurvePatternFeat.Dir2Specified = False
    swFeat = swFeatMgr.CreateFeature(swLocalCurvePatternFeat)

    'Get local curve-driven pattern feature data
    swLocalCurvePatternFeat = swFeat.GetDefinition
    Debug.Print("Local curve-driven pattern properties: ")
    Debug.Print("  Number of components: " & swLocalCurvePatternFeat.GetPatternComponentCount)
    Debug.Print("  Number of skipped items: " & swLocalCurvePatternFeat.GetSkippedItemCount)
    Debug.Print("  Direction 1: ")
    Debug.Print("    Number of pattern instances: " & swLocalCurvePatternFeat.D1InstanceCount)
    Debug.Print("    Is direction flipped: " & swLocalCurvePatternFeat.D1ReverseDirection)
    Debug.Print("    Pattern instances equal spaced: " & swLocalCurvePatternFeat.D1IsEqualSpaced)
    Debug.Print("    Spacing for pattern instances: " & swLocalCurvePatternFeat.D1Spacing)
    Debug.Print("    Curve method: " & swLocalCurvePatternFeat.D1CurveMethod)
    Debug.Print("    Alignment method: " & swLocalCurvePatternFeat.D1AlignmentMethod)
    Debug.Print("  Direction 2:")
    Debug.Print("    Is Direction 2 specified: " & swLocalCurvePatternFeat.Dir2Specified)

```

End Sub

''' <summary>

```
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Local Linear Pattern Example (C#)

This example shows how to create a local linear pattern feature.

```
//-----
// Preconditions: Verify that the assembly exists.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects an edge for Direction 1.
// 3. Selects a subassembly to pattern.
// 4. Creates a LocalLinearPattern1.
// 5. Examine the FeatureManager design tree and graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateLocalLinearPattern_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 swModel;
        private ModelDocExtension swModelDocExt;
        private FeatureManager swFeatureManager;
        private Feature swFeature;
        private LocalLinearPatternFeatureData swLocalLinearPatternFD;
        private bool status;
        private int errors = 0;
        private int warnings = 0;
        private string fileName;
        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\distance linkage.sldasm";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent);
            swModelDocExt = swModel.Extension;
            swFeatureManager = swModel.FeatureManager;
            status = swModelDocExt.SelectByID2("", "EDGE", 0.222723097227572, -0.193386735236572, -0.10172211746567, false, 2, null, null);
            status = swModelDocExt.SelectByID2("mount base-1@distance linkage", "COMPONENT", 0, 0, 0, true, 1, null, 0);
            swLocalLinearPatternFD = (LocalLinearPatternFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmLoc);
            swLocalLinearPatternFD.D1ReverseDirection = true;
            swLocalLinearPatternFD.D1Spacing = 0.1516;
            swLocalLinearPatternFD.D1TotalInstances = 4;
            swLocalLinearPatternFD.D2PatternSeedOnly = false;
            swLocalLinearPatternFD.D2ReverseDirection = false;
            swLocalLinearPatternFD.D2Spacing = 0.05;
            swLocalLinearPatternFD.D2TotalInstances = 1;
            swLocalLinearPatternFD.SynchronizeFlexibleComponents = false;
            swFeature = swFeatureManager.CreateFeature(swLocalLinearPatternFD);
            swModel.ClearSelection2(true);
            swModel.ViewZoomtofit2();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Local Sketch-driven Pattern Example (C#)

This example shows how to create a local sketch-driven pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens specified assembly document.
// 2. Creates a sketch for the local sketch-driven
//    pattern.
// 3. Selects an assembly component and the just-created
//    sketch for the local sketch-driven pattern.
// 4. Creates local sketch-driven pattern.
// 5. Gets values and settings of the local sketch-driven pattern.
// 6. Examine the Immediate window and graphics area.
//
// NOTE: Because this assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FeatureLocalSketchDrivenPatternCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchMgr = default(SketchManager);
            SketchPoint swSketchPoint = default(SketchPoint);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            LocalSketchPatternFeatureData swLocalSketchPatternFeat = default(LocalSketchPatternFeatureData);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            //Open assembly document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\assem1.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);

            //Create sketch
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchMgr.InsertSketch(true);
            swSketchPoint = (SketchPoint)swSketchMgr.CreatePoint(0.025, -0.05, 0.0);
            swSketchPoint = (SketchPoint)swSketchMgr.CreatePoint(0.05, -0.075, 0.0);
            swSketchPoint = (SketchPoint)swSketchMgr.CreatePoint(0.1, -0.05, 0.0);
            swSketchMgr.InsertSketch(true);

            //Select a component and the just-created sketch
            //for the local sketch-driven pattern
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("TestPart1-1@assem1", "COMPONENT", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, true, 16, null, 0);

            //Insert local sketch-driven pattern
            swFeatMgr = (FeatureManager)swModel.FeatureManager;

            swLocalSketchPatternFeat =
            (LocalSketchPatternFeatureData)swFeatMgr.CreateDefinition((int)swLocalSketchPatternReferencePoint_e.swFmLocalSketchPattern);
            swLocalSketchPatternFeat.ReferencePoint = 1; //Bounding box center
            swFeat = (Feature)swFeatMgr.CreateFeature(swLocalSketchPatternFeat);

            //Get local sketch-driven pattern feature data
            swLocalSketchPatternFeat = (LocalSketchPatternFeatureData)swFeat.GetDefinition();
            Debug.Print("Local sketch-driven pattern properties: ");
            Debug.Print("  Number of components: " + swLocalSketchPatternFeat.GetPatternComponentCount());
            Debug.Print("  Number of items skipped: " + swLocalSketchPatternFeat.GetSkippedItemCount());
            Debug.Print("  Type of reference point: " + swLocalSketchPatternFeat.ReferencePoint);
            Debug.Print("  Is reference point a closed curve, sketch point, vertex, or default value: " + swLocalSketchPatternFeat.IsReferencePointValid());

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create Local Sketch-driven Pattern Example (VBA)

This example shows how to create a local sketch-driven pattern feature.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly document.
' 2. Creates a sketch for the local sketch-driven
'    pattern feature.
' 3. Selects an assembly component and the just-created
'    sketch for the local sketch-driven pattern feature.
' 4. Creates the local sketch-driven pattern feature.
' 5. Gets values and settings of the local sketch-driven
'    pattern feature.
' 6. Examine the Immediate window and graphics area.
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As ModelDoc2
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchPoint As SldWorks.SketchPoint
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As FeatureManager
Dim swFeat As Feature
Dim swLocalSketchPatternFeat As SldWorks.LocalSketchPatternFeatureData
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open assembly document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\assem1.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Create sketch
    Set swSketchMgr = swModel.SketchManager
    swSketchMgr.InsertSketch True
    Set swSketchPoint = swSketchMgr.CreatePoint(0.025, -0.05, 0#)
    Set swSketchPoint = swSketchMgr.CreatePoint(0.05, -0.075, 0#)
    Set swSketchPoint = swSketchMgr.CreatePoint(0.1, -0.05, 0#)
    swSketchMgr.InsertSketch True

    'Select a component and the just-created sketch
    'for the local sketch-driven pattern feature
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("TestPart1-1@assem1", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 16, Nothing, 0)

    'Create local sketch-driven pattern feature
    Set swFeatMgr = swModel.FeatureManager
    Set swLocalSketchPatternFeat = swFeatMgr.CreateDefinition(swFmLocalSketchPattern)
    swLocalSketchPatternFeat.ReferencePoint = 1 'Bounding box center
    Set swFeat = swFeatMgr.CreateFeature(swLocalSketchPatternFeat)

    'Get local sketch-driven pattern feature data
    Set swLocalSketchPatternFeat = swFeat.GetDefinition
    Debug.Print "Local sketch-driven pattern properties: "
    Debug.Print "  Number of components: " & swLocalSketchPatternFeat.GetPatternComponentCount
    Debug.Print "  Number of items skipped: " & swLocalSketchPatternFeat.GetSkippedItemCount
    Debug.Print "  Type of reference point as defined in swLocalSketchPatternReferencePoint_e: " &
    swLocalSketchPatternFeat.ReferencePoint

End Sub
```

Create Local Sketch-driven Pattern Example (VB.NET)

This example shows how to create a local sketch-driven pattern feature.

```

-----
Preconditions:
1. Verify that the specified assembly document to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the assembly document.
2. Creates a sketch for the local sketch-driven pattern.
3. Selects an assembly component and the just-created sketch for the local sketch-driven pattern.
4. Creates local sketch-driven pattern.
5. Gets values and settings of the local sketch-driven pattern.
6. Examine the Immediate window.

NOTE: Because this assembly is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSketchMgr As SketchManager
        Dim swSketchPoint As SketchPoint
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatMgr As FeatureManager
        Dim swFeat As Feature
        Dim swLocalSketchPatternFeat As LocalSketchPatternFeatureData
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        'Open assembly document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\assem1.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Create sketch
        swSketchMgr = swModel.SketchManager
        swSketchMgr.InsertSketch(True)
        swSketchPoint = swSketchMgr.CreatePoint(0.025, -0.05, 0.0#)
        swSketchPoint = swSketchMgr.CreatePoint(0.05, -0.075, 0.0#)
        swSketchPoint = swSketchMgr.CreatePoint(0.1, -0.05, 0.0#)
        swSketchMgr.InsertSketch(True)

        'Select a component and the just-created sketch
        'for the local sketch-driven pattern
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("TestPart1-1@assem1", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, True, 10, Nothing, 0)

        'Insert local sketch-driven pattern
        swFeatMgr = swModel.FeatureManager

        swLocalSketchPatternFeat = swFeatMgr.CreateDefinition(swLocalSketchPatternReferencePoint_e.swFmLocalSketchPattern)
        swLocalSketchPatternFeat.ReferencePoint = 1 'Bounding box center
        swFeat = swFeatMgr.CreateFeature(swLocalSketchPatternFeat)

        'Get local sketch-driven pattern feature data
        swLocalSketchPatternFeat = swFeat.GetDefinition
        Debug.Print("Local sketch-driven pattern properties: ")
        Debug.Print(" Number of components: " & swLocalSketchPatternFeat.GetPatternComponentCount)
        Debug.Print(" Number of items skipped: " & swLocalSketchPatternFeat.GetSkippedItemCount)
        Debug.Print(" Type of reference point as defined in
swLocalSketchPatternReferencePoint_e: " & swLocalSketchPatternFeat.ReferencePoint)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Create Loft Body Example (C#)

This example shows how to create a temporary loft body using [IModeler.CreateLoftBody2](#).

```
//-----
// Preconditions:
// 1. Verify that the specified part document template
//    exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Creates and selects sketches of two profiles and
//    a guide curve.
// 3. Creates a temporary loft body.
// 4. Examine the Immediate window and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CreateLoftCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment sketchSegment = default(SketchSegment);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SketchPoint sketchPoint = default(SketchPoint);
            FeatureManager swFeatureMgr = default(FeatureManager);
            RefPlane refPlane = default(RefPlane);
            Feature swFeat = default(Feature);
            bool status = false;
            object profiles = null;
            object guides = null;
            Feature[] profile = new Feature[2];
            Feature[] guideCurve = new Feature[1];
            Modeler swModeler = default(Modeler);
            Body2 swBody = default(Body2);
            int count = 0;
            object[] featArr = null;
            int i = 0;

            //Open new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2014\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Create closed profile
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            sketchSegment = (SketchSegment)swSketchMgr.CreateCircle(0.0, 0.0, 0.0, 0.021796, -0.030937, 0.0);
            sketchPoint = (SketchPoint)swSketchMgr.CreatePoint(0.023454, 0.029699, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            //Create another closed profile
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            refPlane = (RefPlane)swFeatureMgr.InsertRefPlane(8, 0.254, 0, 0, 0, 0);
            status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
            sketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.035118, -0.014102, 0.0, -0.025452, -0.02953, 0.0);
            sketchPoint = (SketchPoint)swSketchMgr.CreatePoint(-0.018033, -0.020392, 0.0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);

            //Create guide curve
            status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 0.0234541440502721, 0.0296993303358475, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -0.0180330744027628, -0.0203923494843098, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 0.0234541440502721, 0.0296993303358475, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -0.0180330744027628, -0.0203923494843098, 0, true, 1, null, 0);
            swModel.Insert3DSplineCurve(false);
            swModel.ClearSelection2(true);

            //Select guide curve and profiles for loft feature
            status = swModelDocExt.SelectByID2("Curve1", "REFERENCECURVES", 0, 0, 0, false, 2, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            Debug.Print("Guide curve name: " + swFeat.Name);
            guideCurve[0] = (Feature)swFeat;
            guides = guideCurve;
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0.00984860021145358, 0.0365397341178567, 0, true, 1, null, 0);
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            Debug.Print("Profile name: " + swFeat.Name);
            profile[0] = (Feature)swFeat;
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -0.0401969362026636, 0.00338231877308527, 0, true, 1, null, 0);
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            Debug.Print("Profile name: " + swFeat.Name);
            profile[1] = (Feature)swFeat;
            profiles = profile;
            swModel.ClearSelection2(true);

            //Create temporary loft body
            swModeler = (Modeler)swApp.GetModeler();
            swBody = (Body2)swModeler.CreateLoftBody2(swModel, profiles, guides, null, false, 0, 0, 0, true, false,
            true, false, true, 1, 1, 1, true, true, 1, 1, false);

            // Test whether the loft body is a temporary body
            status = swBody.IsTemporaryBody();
        }
    }
}
```

```

Debug.Print("Is the loft body a temporary body? " + status);
if (status)
{
    // Display the temporary loft body
    swBody.Display3(swModel, 256, (int)swTempBodySelectOptions_e.swTempBodySelectOptionNone);
    Debug.Print("Temporary loft body displayed; examine the graphics area.");
}
else
{
    Debug.Print("Temporary loft body was not created.");
}

Debug.Print("");

//Verify that the temporary loft body is not a loft feature
//by examining the list of features printed to the
//Immediate window
count = swFeatureMgr.GetFeatureCount(false);
featArr = (Object[])swFeatureMgr.GetFeatures(false);
for (i = 0; i < count - 1; i++)
{
    swFeat = (Feature)featArr[i];
    Debug.Print(swFeat.Name);
}

swModel.ViewZoomtofit2();
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Create Loft Body Example (VBA)

This example shows how to create a temporary loft body using [Modeler::CreateLoftBody2](#).

```

'-----
' Preconditions:
' 1. Verify that the specified part document template
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates and selects sketches of two profiles and
'    a guide curve.
' 3. Creates a temporary loft body.
' 4. Examine the Immediate window and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim sketchSegment As SldWorks.sketchSegment
Dim swSelMgr As SldWorks.SelectionMgr
Dim sketchPoint As SldWorks.sketchPoint
Dim swFeatureMgr As SldWorks.FeatureManager
Dim refPlane As SldWorks.refPlane
Dim swFeat As SldWorks.Feature
Dim status As Boolean
Dim profiles As Variant
Dim guides As Variant
Dim profile(1) As SldWorks.Feature
Dim guideCurve(0) As SldWorks.Feature
Dim swModeler As SldWorks.Modeler
Dim swBody As SldWorks.Body2
Dim count As Long
Dim featArr As Variant
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open new part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Create closed profile
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchMgr = swModel.SketchManager
    Set sketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.021796, -0.030937, 0#)
    Set sketchPoint = swSketchMgr.CreatePoint(0.023454, 0.029699, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Create another closed profile
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set refPlane = swFeatureMgr.InsertRefPlane(8, 0.254, 0, 0, 0, 0)
    status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set sketchSegment = swSketchMgr.CreateCircle(-0.035118, -0.014102, 0#, -0.025452, -0.02953, 0#)
    Set sketchPoint = swSketchMgr.CreatePoint(-0.018033, -0.020392, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Create guide curve
    status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 2.34541440502721E-02, 2.96993303358475E-02, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -1.80330744027628E-02, -2.03923494843098E-02, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 2.34541440502721E-02, 2.96993303358475E-02, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -1.80330744027628E-02, -2.03923494843098E-02, 0, True, 1, Nothing, 0)
    swModel.Insert3DSplineCurve False
    swModel.ClearSelection2 True

    'Select guide curve and profiles for loft feature
    status = swModelDocExt.SelectByID2("Curvel", "REFERENCECURVES", 0, 0, 0, False, 2, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "Guide curve name: " & swFeat.Name
    Set guideCurve(0) = swFeat
    guides = guideCurve
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 9.84860021145358E-03, 3.65397341178567E-02, 0, True, 1, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "Profile name: " & swFeat.Name
    Set profile(0) = swFeat
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -4.01969362026636E-02, 3.38231877308527E-03, 0, True, 1, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "Profile name: " & swFeat.Name
    Set profile(1) = swFeat
    profiles = profile
    swModel.ClearSelection2 True

```

```

'Create temporary loft body
Set swModeler = swApp.GetModeler
Set swBody = swModeler.CreateLoftBody2(swModel, profiles, guides, Nothing, False, 0, 0, 0, True, False, True, False, True, 1, 1, 1, True, True, 1, 1,
False)

' Test whether the loft body is a temporary body
status = swBody.IsTemporaryBody
Debug.Print "Is the loft body a temporary body? " & status
If status Then
    ' Display the temporary loft body
    swBody.Display3 swModel, 256, swTempBodySelectOptions_e.swTempBodySelectOptionNone
    Debug.Print "Temporary loft body displayed; examine the graphics area."
Else
    Debug.Print "Temporary loft body was not created."
End If

Debug.Print ""

'Verify that the temporary loft body is not a loft feature
'by examining the list of features printed to the
'Immediate window
count = swFeatureMgr.GetFeatureCount(False)
featArr = swFeatureMgr.GetFeatures(False)
For i = 0 To count - 1
    Set swFeat = featArr(i)
    Debug.Print swFeat.Name
Next i

swModel.ViewZoomtofit2

End Sub

```

Create Loft Body Example (VB.NET)

This example shows how to create a temporary loft body using [Modeler.CreateLoftBody2](#).

```

-----
Preconditions:
1. Verify that the specified part document template exists.
2. Open the Immediate window.
Postconditions:
1. Opens a new part document.
2. Creates and selects sketches of two profiles and a guide curve.
3. Creates a temporary loft body.
4. Examine the Immediate window and graphics area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim sketchSegment As SketchSegment
        Dim swSelMgr As SelectionMgr
        Dim sketchPoint As SketchPoint
        Dim swFeatureMgr As FeatureManager
        Dim refPlane As RefPlane
        Dim swFeat As Feature
        Dim status As Boolean
        Dim profiles As Object
        Dim guides As Object
        Dim profile(1) As Feature
        Dim guideCurve(0) As Feature
        Dim swModeler As Modeler
        Dim swBody As Body2
        Dim count As Integer
        Dim featArr As Object
        Dim i As Integer

        'Open new part document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension

        'Create closed profile
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchMgr = swModel.SketchManager
        sketchSegment = swSketchMgr.CreateCircle(0.0#, 0.0#, 0.0#, 0.021796, -0.030937, 0.0#)
        sketchPoint = swSketchMgr.CreatePoint(0.023454, 0.029699, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        'Create another closed profile
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        refPlane = swFeatureMgr.InsertRefPlane(8, 0.254, 0, 0, 0, 0)
        status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        sketchSegment = swSketchMgr.CreateCircle(-0.035118, -0.014102, 0.0#, -0.025452, -0.02953, 0.0#)
        sketchPoint = swSketchMgr.CreatePoint(-0.018033, -0.020392, 0.0#)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

        'Create guide curve
        status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 0.0234541440502721, 0.0296993303358475, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -0.0180330744027628, -0.0203923494843098, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 0.0234541440502721, 0.0296993303358475, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -0.0180330744027628, -0.0203923494843098, 0, True, 1, Nothing, 0)
        swModel.Insert3DSplineCurve(False)
        swModel.ClearSelection2(True)

        'Select guide curve and profiles for loft feature
        status = swModelDocExt.SelectByID2("Curve1", "REFERENCECURVES", 0, 0, 0, False, 2, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        Debug.Print("Guide curve name: " & swFeat.Name)
        guideCurve(0) = swFeat
        guides = guideCurve
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0.00984860021145358, 0.0365397341178567, 0, True, 1, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        Debug.Print("Profile name: " & swFeat.Name)
        profile(0) = swFeat
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -0.0401969362026636, 0.00338231877308527, 0, True, 1, Nothing, 0)
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        Debug.Print("Profile name: " & swFeat.Name)
        profile(1) = swFeat
        profiles = profile
        swModel.ClearSelection2(True)

        'Create temporary loft body
        swModeler = swApp.GetModeler
        swBody = swModeler.CreateLoftBody2(swModel, profiles, guides, Nothing, False, 0, 0, 0, True, False, True, False, True, 1, 1, 1, True, True, 1, 1,

False)

        ' Test whether the loft body is a temporary body
        status = swBody.IsTemporaryBody
        Debug.Print("Is the loft body a temporary body? " & status)
        If status Then
            ' Display the temporary loft body
            swBody.Display3(swModel, 256, swTempBodySelectOptions_e.swTempBodySelectOptionNone)
            Debug.Print("Temporary loft body displayed; examine the graphics area.")
        Else
    
```

```

        Debug.Print("Temporary loft body was not created.")
    End If

    Debug.Print("")

    'Verify that the temporary loft body is not a loft feature
    'by examining the list of features printed to the
    'Immediate window
    count = swFeatureMgr.GetFeatureCount(False)
    featArr = swFeatureMgr.GetFeatures(False)
    For i = 0 To count - 1
        swFeat = featArr(i)
        Debug.Print(swFeat.Name)
    Next i

    swModel.ViewZoomtofit2()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Create Loft Surface Body and Change Into Feature Example (VBA)

This example shows how to create a loft surface body and change that body into a feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part document template
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Creates and selects sketches of two profiles and
'    a guide curve.
' 3. Creates a loft surface body.
' 4. Turns the loft surface body into a feature.
' 5. Examine the FeatureManager design tree and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim sketchSegment As SldWorks.sketchSegment
Dim swSelMgr As SldWorks.SelectionMgr
Dim sketchPoint As SldWorks.sketchPoint
Dim swFeatureMgr As SldWorks.FeatureManager
Dim refPlane As SldWorks.refPlane
Dim swFeat As SldWorks.Feature
Dim status As Boolean
Dim profiles As Variant
Dim guides As Variant
Dim profile(1) As SldWorks.Feature
Dim guideCurve(0) As SldWorks.Feature
Dim swModeler As SldWorks.Modeler
Dim swBody As SldWorks.Body2
Dim swPart As PartDoc
Dim vFeatures As Variant

Sub main()
    Set swApp = Application.SldWorks

    'Open new part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension

    'Create closed profile
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchMgr = swModel.SketchManager
    Set sketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.021796, -0.030937, 0#)
    Set sketchPoint = swSketchMgr.CreatePoint(0.023454, 0.029699, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Create another closed profile
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set refPlane = swFeatureMgr.InsertRefPlane(8, 0.254, 0, 0, 0, 0)
    status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set sketchSegment = swSketchMgr.CreateCircle(-0.035118, -0.014102, 0#, -0.025452, -0.02953, 0#)
    Set sketchPoint = swSketchMgr.CreatePoint(-0.018033, -0.020392, 0#)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

    'Create guide curve
    status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 2.34541440502721E-02, 2.96993303358475E-02, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -1.80330744027628E-02, -2.03923494843098E-02, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Point4@Sketch1", "EXTSKETCHPOINT", 2.34541440502721E-02, 2.96993303358475E-02, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Point5@Sketch2", "EXTSKETCHPOINT", -1.80330744027628E-02, -2.03923494843098E-02, 0, True, 1, Nothing, 0)
    swModel.Insert3DSplineCurve False
    swModel.ClearSelection2 True

    'Select guide curve and profiles for loft feature
    status = swModelDocExt.SelectByID2("Curvel", "REFERENCECURVES", 0, 0, 0, False, 2, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set guideCurve(0) = swFeat
    guides = guideCurve
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 9.84860021145358E-03, 3.65397341178567E-02, 0, True, 1, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set profile(0) = swFeat
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -4.01969362026636E-02, 3.38231877308527E-03, 0, True, 1, Nothing, 0)
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set profile(1) = swFeat
    profiles = profile
    swModel.ClearSelection2 True

    'Create loft surface body
    Set swModeler = swApp.GetModeler
    Set swBody = swModeler.CreateLoftBody2(swModel, profiles, guides, Nothing, False, 0, 0, 0, True, False, True, False, False, 1, 1, 1, True, True, 1, 1, False)
End Sub

```

```
'Turn loft surface body into a feature
Set swPart = swModel
vFeatures = swPart.CreateSurfaceFeatureFromBody(swBody, swCreateFeatureBodyOpts_e.swCreateFeatureBodyCheck)

'Update the FeatureManager design tree
swModel.EditRebuild3

'Update graphics
swModel.ViewZoomtofit2

End Sub
```

Create Machining Costing Analysis Example (C#)

This example shows how to create a machining Costing analysis.

```
//-----
// Preconditions:
// 1. Open:
// C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\costing\machined_part.sldprt
// 2. Verify that the Costing templates exist by clicking Tools > Options >
// System Options > File Locations and select Costing templates in
// Show folders for in SOLIDWORKS Design. Click Cancel to close the dialog.
// 3. Add a reference to SolidWorks.Interop.sldcostingapi.dll.
// 4. Open the Immediate window.
// 5. Run the macro.
//
// Postconditions:
// 1. Creates machining Costing analysis, which
// might take one or more minutes to complete.
// 2. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.sldcostingapi;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace SOLIDWORKSMachiningCostingCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            CostManager swCosting = default(CostManager);
            CostPart swCostingPart = default(CostPart);
            CostBody swCostingBody = default(CostBody);
            object swCostingModel = null;
            CostAnalysis swCostingAnalysis = default(CostAnalysis);
            CostAnalysisMachining swCostingMachining = default(CostAnalysisMachining);
            string machiningCostingTemplatePathName = null;
            string machiningCostingReportTemplateFolderName = null;
            int nbrMachiningCostingTemplate = 0;
            int nbrCommonCostingTemplate = 0;
            object[] commonCostingTemplates = null;
            object[] machiningCostingTemplates = null;
            int nbrCostingBodies = 0;
            object[] costingBodies = null;
            string costingBodyName = null;
            int i = 0;
            bool isBody = false;
            CostFeature swCostingFeat = default(CostFeature);
            CostFeature swCostingNextFeat = default(CostFeature);
            CostFeature swCostingSubFeat = default(CostFeature);
            CostFeature swCostingNextSubFeat = default(CostFeature);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Get Costing templates names
            Debug.Print("Costing template folders:");
            machiningCostingTemplatePathName = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swFileLocationsCos
            Debug.Print("    Name of Costing template folder: " + machiningCostingTemplatePathName);
            machiningCostingReportTemplateFolderName = swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swFileLoca
            Debug.Print("    Name of Costing report template folder: " + machiningCostingReportTemplateFolderName);
            Debug.Print("");

            // Get CostingManager
            swCosting = (CostManager)swModelDocExt.GetCostingManager();
            swCosting.WaitForUIUpdate();

            // Get the number of templates
            nbrMachiningCostingTemplate = swCosting.GetTemplateCount((int)swcCostingType_e.swcCostingType_Machining);
            nbrCommonCostingTemplate = swCosting.GetTemplateCount((int)swcCostingType_e.swcCostingType_Common);

            // Get names of templates
            machiningCostingTemplates = (object[])swCosting.GetTemplatePathnames((int)swcCostingType_e.swcCostingType_Machining);
```

```

commonCostingTemplates = (object[])swCosting.GetTemplatePathnames((int)swCostingType_e.swCostingType_Common);

Array.Resize(ref machiningCostingTemplates, nbrMachiningCostingTemplate + 1);
Array.Resize(ref commonCostingTemplates, nbrCommonCostingTemplate + 1);

Debug.Print("Costing templates:");

// Print names of templates to Immediate window
for (i = 0; i <= (nbrMachiningCostingTemplate - 1); i++)
{
    Debug.Print("    Name of machining Costing template: " + machiningCostingTemplates[i]);
}

Debug.Print("");

for (i = 0; i <= (nbrCommonCostingTemplate - 1); i++)
{
    Debug.Print("    Name of common Costing template: " + commonCostingTemplates[i]);
}

Debug.Print("");

// Get Costing part
swCostingModel = (object)swCosting.CostingModel;
swCostingPart = (CostPart)swCostingModel;

// Create common Costing analysis
swCostingAnalysis = (CostAnalysis)swCostingPart.CreateCostAnalysis("c:\\program files\\solidworks corp\\solidworks\\lang\\e

// Get common Costing analysis data
Debug.Print("Common Costing analysis data:");
Debug.Print("    Template name: " + swCostingAnalysis.CostingTemplateName);
Debug.Print("    Currency code: " + swCostingAnalysis.CurrencyCode);
Debug.Print("    Currency name: " + swCostingAnalysis.CurrencyName);
Debug.Print("    Currency separator: " + swCostingAnalysis.CurrencySeparator);
Debug.Print("    Total manufacturing cost: " + swCostingAnalysis.GetManufacturingCost());
Debug.Print("    Material costs: " + swCostingAnalysis.GetMaterialCost());
Debug.Print("    Setup cost: " + swCostingAnalysis.GetSetupCost());
Debug.Print("    Total cost to charge: " + swCostingAnalysis.GetTotalCostToCharge());
Debug.Print("    Total cost to manufacture: " + swCostingAnalysis.GetTotalCostToManufacture());
Debug.Print("    Lot size: " + swCostingAnalysis.LotSize);
Debug.Print("    Total quantity: " + swCostingAnalysis.TotalQuantity);
Debug.Print("");

// Get Costing bodies
nbrCostingBodies = swCostingPart.GetBodyCount();
if ((nbrCostingBodies > 0))
{
    Debug.Print("Costing bodies:");
    Debug.Print("    Number of Costing bodies: " + nbrCostingBodies);
    costingBodies = (object[])swCostingPart.GetBodies();
    for (i = 0; i <= (nbrCostingBodies - 1); i++)
    {
        swCostingBody = (CostBody)costingBodies[i];
        costingBodyName = swCostingBody.GetName();
        Debug.Print("    Name of Costing body: " + costingBodyName);
        // Make sure body is machining body
        if ((swCostingBody.GetBodyType() == (int)swcBodyType_e.swcBodyType_Machined))
        {
            isBody = true;
            // Determine analysis status of Costing body
            switch ((int)swCostingBody.BodyStatus)
            {
                case (int)swcBodyStatus_e.swcBodyStatus_NotAnalysed:
                    // Create Costing analysis
                    swCostingAnalysis = (CostAnalysis)swCostingBody.CreateCostAnalysis("c:\\program files\\solidworks corp\\
                    Debug.Print("    Creating machining Costing analysis for: " + swCostingBody.GetName());
                    break;
                case (int)swcBodyStatus_e.swcBodyStatus_Analysed:
                    // Get Costing analysis
                    swCostingAnalysis = (CostAnalysis)swCostingBody.GetCostAnalysis();
                    Debug.Print("    Getting machining Costing analysis for: " + swCostingBody.GetName());
                    break;
                case (int)swcBodyStatus_e.swcBodyStatus_Excluded:
                    // Body excluded from Costing analysis
                    Debug.Print("    Excluded from machining Costing analysis: " + swCostingBody.GetName());
                    break;
                case (int)swcBodyStatus_e.swcBodyStatus_AssignedCustomCost:
                    // Body has an assigned custom Cost
                    Debug.Print("    Custom cost assigned: " + swCostingBody.GetName());
                    break;
            }
        }

        Debug.Print("");
    }
}
}

if (!isBody)

```

```

{
    Debug.Print ("");
    Debug.Print ("No bodies in part! Exiting macro.");
    return;
}

// Get machining Costing Analysis data
swCostingMachining = (CostAnalysisMachining)swCostingAnalysis.GetSpecificAnalysis();
Debug.Print("Machining Costing analysis: ");
Debug.Print("    Current material: " + swCostingMachining.CurrentMaterial);
Debug.Print("    Current material class: " + swCostingMachining.CurrentMaterialClass);
Debug.Print("    Current plate thickness: " + swCostingMachining.CurrentPlateThickness);

Debug.Print("");

// Get Costing features
Debug.Print("Costing features:");
swCostingFeat = (CostFeature)swCostingAnalysis.GetFirstCostFeature();
while ((swCostingFeat != null))
{
    Debug.Print("    Feature: " + swCostingFeat.Name);
    Debug.Print("    Type: " + swCostingFeat.GetType());
    Debug.Print("    Setup related: " + swCostingFeat.IsSetup);
    Debug.Print("    Overridden: " + swCostingFeat.IsOverridden);
    Debug.Print("    Combined cost: " + swCostingFeat.CombinedCost);
    Debug.Print("    Combined time: " + swCostingFeat.CombinedTime);

    swCostingSubFeat = swCostingFeat.GetFirstSubFeature();
    while ((swCostingSubFeat != null))
    {
        Debug.Print("        Subfeature: " + swCostingSubFeat.Name);
        Debug.Print("        Type: " + swCostingSubFeat.GetType());
        Debug.Print("        Setup related: " + swCostingSubFeat.IsSetup);
        Debug.Print("        Overridden: " + swCostingSubFeat.IsOverridden);
        Debug.Print("        Combined cost: " + swCostingSubFeat.CombinedCost);
        Debug.Print("        Combined time: " + swCostingSubFeat.CombinedTime);
        swCostingNextSubFeat = (CostFeature)swCostingSubFeat.GetNextFeature();
        swCostingSubFeat = null;
        swCostingSubFeat = (CostFeature)swCostingNextSubFeat;
        swCostingNextSubFeat = null;
    }
    swCostingNextFeat = swCostingFeat.GetNextFeature();
    swCostingFeat = null;
    swCostingFeat = (CostFeature)swCostingNextFeat;
    swCostingNextFeat = null;
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Create Machining Costing Analysis Example (VBA)

This example shows how to create a machining Costing analysis.

```

'-----
' Preconditions:
' 1. Open:
'   C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\costing\machined_part.sldprt
' 2. Verify that the Costing templates exist by clicking Tools > Options >
'   System Options > File Locations and selecting Costing templates in
'   Show folders for in SOLIDWORKS Design. Click Cancel to close the dialog.
' 3. Add a reference to SldCosting API 20nn Type Library.
' 4. Open the Immediate window.
' 5. Run the macro.
'
' Postconditions:
' 1. Creates machining Costing analysis, which might
'   take one or more minutes to complete.
' 2. Examine the Immediate window
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swCosting As SldCostingAPI.CostManager
Dim swCostingPart As SldCostingAPI.CostPart
Dim swCostingBody As SldCostingAPI.CostBody
Dim swCostingModel As Object
Dim swCostingAnalysis As SldCostingAPI.CostAnalysis
Dim swCostingMachining As SldCostingAPI.CostAnalysisMachining
Dim machiningCostingTemplateName As String
Dim machiningCostingReportTemplateFolderName As String
Dim nbrMachiningCostingTemplate As Long
Dim nbrCommonCostingTemplate As Long
Dim commonCostingTemplates As Variant
Dim machiningCostingTemplates As Variant
Dim nbrCostingBodies As Long
Dim costingBodies As Variant
Dim costingBodyName As String
Dim i As Long
Dim isBody As Boolean
Dim swCostingFeat As SldCostingAPI.CostFeature
Dim swCostingNextFeat As SldCostingAPI.CostFeature
Dim swCostingSubFeat As SldCostingAPI.CostFeature
Dim swCostingNextSubFeat As SldCostingAPI.CostFeature

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    ' Get Costing templates names
    Debug.Print ("Costing template folders:")
    machiningCostingTemplateName = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCostingTemplates)
    Debug.Print "    Name of Costing template folder: " & machiningCostingTemplateName
    machiningCostingReportTemplateFolderName =
    swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCostingReportTemplateFolder)
    Debug.Print "    Name of Costing report template folder: " & machiningCostingReportTemplateFolderName
    Debug.Print ""

    ' Get CostingManager
    Set swCosting = swModelDocExt.GetCostingManager
    swCosting.WaitForUIUpdate

    ' Get the number of templates
    nbrMachiningCostingTemplate = swCosting.GetTemplateCount(swcCostingType_e.swcCostingType_Machining)
    nbrCommonCostingTemplate = swCosting.GetTemplateCount(swcCostingType_e.swcCostingType_Common)

    ' Get names of templates
    machiningCostingTemplates = swCosting.GetTemplatePathnames(swcCostingType_e.swcCostingType_Machining)
    commonCostingTemplates = swCosting.GetTemplatePathnames(swcCostingType_e.swcCostingType_Common)

    ReDim Preserve machiningCostingTemplates(nbrMachiningCostingTemplate)
    ReDim Preserve commonCostingTemplates(nbrCommonCostingTemplate)

    Debug.Print ("Costing templates:")

    ' Print names of templates to Immediate window
    For i = 0 To (UBound(machiningCostingTemplates) - 1)
        Debug.Print "    Name of machining Costing template: " & machiningCostingTemplates(i)
    Next i

    Debug.Print ""

    For i = 0 To (UBound(commonCostingTemplates) - 1)
        Debug.Print "    Name of common Costing template: " & commonCostingTemplates(i)
    Next i

    Debug.Print ""

    ' Get Costing part
    Set swCostingModel = swCosting.CostingModel

```

```

Set swCostingPart = swCostingModel

' Create common Costing analysis
Set swCostingAnalysis = swCostingPart.CreateCostAnalysis("c:\program files\solidworks corp\solidworks\lang\english\costing
templates\multibodytemplate_default(englishstandard).sldctc")

' Get common Costing analysis data
Debug.Print ("Common Costing analysis data:")
Debug.Print ("    Template name: " & swCostingAnalysis.CostingTemplateName)
Debug.Print ("    Currency code: " & swCostingAnalysis.CurrencyCode)
Debug.Print ("    Currency name: " & swCostingAnalysis.CurrencyName)
Debug.Print ("    Currency separator: " & swCostingAnalysis.CurrencySeparator)
Debug.Print ("    Total manufacturing cost: " & swCostingAnalysis.GetManufacturingCost)
Debug.Print ("    Material costs: " & swCostingAnalysis.GetMaterialCost)
Debug.Print ("    Setup cost: " & swCostingAnalysis.GetSetupCost)
Debug.Print ("    Total cost to charge: " & swCostingAnalysis.GetTotalCostToCharge)
Debug.Print ("    Total cost to manufacture: " & swCostingAnalysis.GetTotalCostToManufacture)
Debug.Print ("    Lot size: " & swCostingAnalysis.LotSize)
Debug.Print ("    Total quantity: " & swCostingAnalysis.TotalQuantity)
Debug.Print ""

' Get Costing bodies
nbrCostingBodies = swCostingPart.GetBodyCount
isBody = False
If (nbrCostingBodies > 0) Then
    Debug.Print "Costing bodies:"
    Debug.Print ("    Number of Costing bodies: " & nbrCostingBodies)
    costingBodies = swCostingPart.GetBodies
    For i = 0 To (nbrCostingBodies - 1)
        Set swCostingBody = costingBodies(i)
        costingBodyName = swCostingBody.GetName
        Debug.Print ("    Name of Costing body: " & costingBodyName)
        ' Make sure body is machining body
        If (swCostingBody.GetBodyType = swcBodyType_e.swcBodyType_Machined) Then
            isBody = True
            ' Determine analysis status of Costing body
            Select Case swCostingBody.BodyStatus
                Case swcBodyStatus_e.swcBodyStatus_NotAnalysed
                    ' Create Costing analysis
                    Set swCostingAnalysis = swCostingBody.CreateCostAnalysis("c:\program files\solidworks
corp\solidworks\lang\english\costing templates\machiningtemplate_default(englishstandard).sldcts")
                    Debug.Print ("        Creating machining Costing analysis for: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_Analysed
                    ' Get Costing analysis
                    Set swCostingAnalysis = swCostingBody.GetCostingAnalysis
                    Debug.Print ("        Getting machining Costing analysis for: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_Excluded
                    ' Body excluded from Costing analysis
                    Debug.Print ("        Excluded from machining Costing analysis: " & swCostingBody.GetName)
                Case swcBodyStatus_e.swcBodyStatus_AssignedCustomCost
                    ' Body has an assigned custom Cost
                    Debug.Print ("        Custom cost assigned: " & swCostingBody.GetName)
            End Select
        End If
    Next i
End If

If Not isBody Then
    Debug.Print ("")
    Debug.Print ("No bodies in part! Exiting macro.")
    Exit Sub
End If

' Get machining Costing Analysis data
Set swCostingMachining = swCostingAnalysis.GetSpecificAnalysis
Debug.Print "Machining Costing analysis: "
Debug.Print ("    Current material: " & swCostingMachining.CurrentMaterial)
Debug.Print ("    Current material class: " & swCostingMachining.CurrentMaterialClass)
Debug.Print ("    Current plate thickness: " & swCostingMachining.CurrentPlateThickness)

Debug.Print ("")

' Get Costing features
Debug.Print "Costing features:"
Set swCostingFeat = swCostingAnalysis.GetFirstCostFeature
While Not swCostingFeat Is Nothing
    Debug.Print ("    Feature: " & swCostingFeat.Name)
    Debug.Print ("    Type: " & swCostingFeat.GetType)
    Debug.Print ("    Setup related: " & swCostingFeat.IsSetup)
    Debug.Print ("    Overridden: " & swCostingFeat.IsOverridden)
    Debug.Print ("    Combined cost: " & swCostingFeat.CombinedCost)
    Debug.Print ("    Combined time: " & swCostingFeat.CombinedTime)

    Set swCostingSubFeat = swCostingFeat.GetFirstSubFeature
    While Not swCostingSubFeat Is Nothing
        Debug.Print ("        Subfeature: " & swCostingSubFeat.Name)
        Debug.Print ("        Type: " & swCostingSubFeat.GetType)
        Debug.Print ("        Setup related: " & swCostingSubFeat.IsSetup)
        Debug.Print ("        Overridden: " & swCostingSubFeat.IsOverridden)
        Debug.Print ("        Combined cost: " & swCostingSubFeat.CombinedCost)
        Debug.Print ("        Combined time: " & swCostingSubFeat.CombinedTime)
        Set swCostingNextSubFeat = swCostingSubFeat.GetNextFeature
        Set swCostingSubFeat = Nothing
        Set swCostingSubFeat = swCostingNextSubFeat
        Set swCostingNextSubFeat = Nothing
    Wend
    Set swCostingNextFeat = swCostingFeat.GetNextFeature
    Set swCostingFeat = Nothing
    Set swCostingFeat = swCostingNextFeat
    Set swCostingNextFeat = Nothing
End While

```

Wend

End Sub

Create Machining Costing Analysis Example (VB.NET)

This example shows how to create a machining Costing analysis.

```

'-----
' Preconditions:
' 1. Open:
'    C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\costing\machined_part.sldprt
' 2. Verify that the Costing templates exist by clicking Tools > Options
'    System Options > File Locations and selecting Costing templates in
'    Show folders for in SOLIDWORKS Design. Click Cancel to close the dialog.
' 3. Add a reference to SolidWorks.Interop.sldcostingapi.dll.
' 4. Open the Immediate window.
' 5. Run the macro.
'
' Postconditions:
' 1. Creates machining Costing analysis, which might
'    take one or more minutes to complete.
' 2. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.sldcostingapi
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swCosting As CostManager
        Dim swCostingPart As CostPart
        Dim swCostingBody As CostBody
        Dim swCostingModel As Object
        Dim swCostingAnalysis As CostAnalysis
        Dim swCostingMachining As CostAnalysisMachining
        Dim machiningCostingTemplatePathName As String
        Dim machiningCostingReportTemplateFolderName As String
        Dim nbrMachiningCostingTemplate As Integer
        Dim nbrCommonCostingTemplate As Integer
        Dim commonCostingTemplates As Object
        Dim machiningCostingTemplates As Object
        Dim nbrCostingBodies As Integer
        Dim costingBodies As Object
        Dim costingBodyName As String
        Dim i As Integer
        Dim isBody As Boolean
        Dim swCostingFeat As CostFeature
        Dim swCostingNextFeat As CostFeature
        Dim swCostingSubFeat As CostFeature
        Dim swCostingNextSubFeat As CostFeature

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        ' Get Costing templates names
        Debug.Print("Costing template folders:")
        machiningCostingTemplatePathName = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCostingTempl
        Debug.Print("    Name of Costing template folder: " & machiningCostingTemplatePathName)
        machiningCostingReportTemplateFolderName = swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swFileLocationsCost
        Debug.Print("    Name of Costing report template folder: " & machiningCostingReportTemplateFolderName)
        Debug.Print("")

        ' Get CostingManager
        swCosting = swModelDocExt.GetCostingManager
        swCosting.WaitForUIUpdate()

        ' Get the number of templates
        nbrMachiningCostingTemplate = swCosting.GetTemplateCount(swCostingType_e.swcCostingType_Machining)
        nbrCommonCostingTemplate = swCosting.GetTemplateCount(swCostingType_e.swcCostingType_Common)

        ' Get names of templates
        machiningCostingTemplates = swCosting.GetTemplatePathnames(swCostingType_e.swcCostingType_Machining)
        commonCostingTemplates = swCosting.GetTemplatePathnames(swCostingType_e.swcCostingType_Common)

        ReDim Preserve machiningCostingTemplates(nbrMachiningCostingTemplate)
        ReDim Preserve commonCostingTemplates(nbrCommonCostingTemplate)

        Debug.Print("Costing templates:")

        ' Print names of templates to Immediate window
        For i = 0 To (UBound(machiningCostingTemplates) - 1)
            Debug.Print("    Name of machining Costing template: " & machiningCostingTemplates(i))
        
```

```

Next i

Debug.Print("")

For i = 0 To (UBound(commonCostingTemplates) - 1)
    Debug.Print("    Name of common Costing template: " & commonCostingTemplates(i))
Next i

Debug.Print("")

' Get Costing part
swCostingModel = swCosting.CostingModel
swCostingPart = swCostingModel

' Create common Costing analysis
swCostingAnalysis = swCostingPart.CreateCostAnalysis("c:\program files\solidworks corp\solidworks\lang\english\costing template

' Get common Costing analysis data
Debug.Print("Common Costing analysis data:")
Debug.Print("    Template name: " & swCostingAnalysis.costingTemplateName)
Debug.Print("    Currency code: " & swCostingAnalysis.CurrencyCode)
Debug.Print("    Currency name: " & swCostingAnalysis.CurrencyName)
Debug.Print("    Currency separator: " & swCostingAnalysis.CurrencySeparator)
Debug.Print("    Total manufacturing cost: " & swCostingAnalysis.GetManufacturingCost)
Debug.Print("    Material costs: " & swCostingAnalysis.GetMaterialCost)
Debug.Print("    Setup cost: " & swCostingAnalysis.GetSetupCost)
Debug.Print("    Total cost to charge: " & swCostingAnalysis.GetTotalCostToCharge)
Debug.Print("    Total cost to manufacture: " & swCostingAnalysis.GetTotalCostToManufacture)
Debug.Print("    Lot size: " & swCostingAnalysis.LotSize)
Debug.Print("    Total quantity: " & swCostingAnalysis.TotalQuantity)
Debug.Print("")

' Get Costing bodies
nbrCostingBodies = swCostingPart.GetBodyCount
isBody = False
If (nbrCostingBodies > 0) Then
    Debug.Print("Costing bodies:")
    Debug.Print("    Number of Costing bodies: " & nbrCostingBodies)
    costingBodies = swCostingPart.GetBodies
    For i = 0 To (nbrCostingBodies - 1)
        swCostingBody = costingBodies(i)
        costingBodyName = swCostingBody.GetName
        Debug.Print("    Name of Costing body: " & costingBodyName)
        ' Make sure body is machining body
        If (swCostingBody.GetBodyType = swcBodyType_e.swcBodyType_Machined) Then
            isBody = True
            ' Determine analysis status of Costing body
            Select Case swCostingBody.BodyStatus
            Case swcBodyStatus_e.swcBodyStatus_NotAnalysed
                ' Create Costing analysis
                swCostingAnalysis = swCostingBody.CreateCostAnalysis("c:\program files\solidworks corp\solidworks\lang\engl
                Debug.Print("    Creating machining Costing analysis for: " & swCostingBody.GetName)
            Case swcBodyStatus_e.swcBodyStatus_Analysed
                ' Get Costing analysis
                swCostingAnalysis = swCostingBody.GetCostAnalysis
                Debug.Print("    Getting machining Costing analysis for: " & swCostingBody.GetName)
            Case swcBodyStatus_e.swcBodyStatus_Excluded
                ' Body excluded from Costing analysis
                Debug.Print("    Excluded from machining Costing analysis: " & swCostingBody.GetName)
            Case swcBodyStatus_e.swcBodyStatus_AssignedCustomCost
                ' Body has an assigned custom Cost
                Debug.Print("    Custom cost assigned: " & swCostingBody.GetName)
            End Select
        End Select
        Debug.Print("")
    Next i
End If

If Not isBody Then
    Debug.Print("")
    Debug.Print("No bodies in part! Exiting macro.")
    Exit Sub
End If

' Get machining Costing Analysis data
swCostingMachining = swCostingAnalysis.GetSpecificAnalysis
Debug.Print("Machining Costing analysis: ")
Debug.Print("    Current material: " & swCostingMachining.CurrentMaterial)
Debug.Print("    Current material class: " & swCostingMachining.CurrentMaterialClass)
Debug.Print("    Current plate thickness: " & swCostingMachining.CurrentPlateThickness)

Debug.Print("")

' Get Costing features
Debug.Print("Costing features:")
swCostingFeat = swCostingAnalysis.GetFirstCostFeature
While Not swCostingFeat Is Nothing
    Debug.Print("    Feature: " & swCostingFeat.Name)
    Debug.Print("    Type: " & swCostingFeat.GetType)
    Debug.Print("    Setup related: " & swCostingFeat.IsSetup)
    Debug.Print("    Overridden: " & swCostingFeat.IsOverridden)
    Debug.Print("    Combined cost: " & swCostingFeat.CombinedCost)
    Debug.Print("    Combined time: " & swCostingFeat.CombinedTime)

```

```

swCostingSubFeat = swCostingFeat.GetFirstSubFeature
While Not swCostingSubFeat Is Nothing
    Debug.Print("      Subfeature: " & swCostingSubFeat.Name)
    Debug.Print("      Type: " & swCostingSubFeat.GetType)
    Debug.Print("      Setup related: " & swCostingSubFeat.IsSetup)
    Debug.Print("      Overridden: " & swCostingSubFeat.IsOverridden)
    Debug.Print("      Combined cost: " & swCostingSubFeat.CombinedCost)
    Debug.Print("      Combined time: " & swCostingSubFeat.CombinedTime)
    swCostingNextSubFeat = swCostingSubFeat.GetNextFeature
    swCostingSubFeat = Nothing
    swCostingSubFeat = swCostingNextSubFeat
    swCostingNextSubFeat = Nothing
End While
swCostingNextFeat = swCostingFeat.GetNextFeature
swCostingFeat = Nothing
swCostingFeat = swCostingNextFeat
swCostingNextFeat = Nothing
End While

```

```

End Sub

```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

```

End Class

```